

문제 6

Kaggle 형 train_prob.csv로 문제 target을 예측하는 모델을 만들고,

test_prob.csv에 대한 target 예측하여 다음과 같은 형식의 answer6.csv를 만들어라.

id, target

0, 6.9

5, 7.8

...

평가지표

$$RMSE(Y, \hat{Y}) = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

강사: 멀티캠퍼스 강선구(sunku0316.kang@multicampus.com, sun9sun9@gmail.com)

```
In [1]: # 실행 환경 확인

import pandas as pd
import numpy as np
import sklearn
import scipy
import statsmodels
import mlxtend
import sys
import xgboost as xgb

print(sys.version)
for i in [pd, np, sklearn, scipy, mlxtend, statsmodels, xgb]:
    print(i.__name__, i.__version__)
```

```
3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 20:34:20) [MSC v.1916 64 bit (AMD64)]
pandas 0.25.1
numpy 1.18.5
sklearn 0.21.3
scipy 1.5.2
mlxtend 0.15.0.0
statsmodels 0.11.1
xgboost 0.80
```

```
In [2]: df_train = pd.read_csv('train_prob.csv', index_col='id')
df_test = pd.read_csv('test_prob.csv', index_col='id')
df_ans = pd.read_csv('test_prob_ans.csv', index_col='id')
```

```
In [3]: '''
코드는 `df_train`과 `df_test` 데이터프레임에서 특정 범주형 열의 값을 지정된 매핑에 따라
치환(replace)하고, 그 결과를 확인하는 작업을 수행합니다.
이를 통해 데이터의 전처리를 진행합니다.
```

1. 치환 리스트 정의

`repl\_list`는 치환할 열, 치환할 값의 매핑, 치환 후 값의 예상 개수를 정의합니다.

2. 반복문을 통한 치환 및 검증

각 범주형 열에 대해 치환을 수행하고, 치환 결과를 검증합니다.

내부 동작 설명

1. 변수와 매핑 출력

현재 처리 중인 열(`v`), 매핑(`d`), 예상 개수(`cnts`)를 출력합니다.

2. 치환 후 결과 저장

`df\_train`의 열 `v`에서 매핑(`d`)에 따라 값을 치환한 결과를 `s\_repl`에 저장합니다.

3. 치환 결과 검증

- `s\_repl.nunique() != len(cnts)`:
치환 후 고유 값의 개수가 예상 개수와 다르면 검증 실패.
- `(s\_repl.value\_counts().sort\_index() != cnts).any()`:
각 고유 값의 개수가 예상 개수와 다르면 검증 실패.
- 검증에 실패하면 치환 후의 값 개수를 출력하고 반복문을 종료합니다.

4. 치환 결과 적용

검증에 성공하면 `df\_train`과 `df\_test`의 열 `v`에서 매핑(`d`)에 따라 값을 치환한 결과를 적용합니다.

3. 범주형 및 연속형 특성 목록 정의

치환 작업 후, 범주형 및 연속형 특성의 목록을 정의합니다.

- `cat\_cols`: 'cat0'부터 'cat9'까지의 범주형 특성 목록을 생성합니다.

- `cont\_cols`: 'cont0'부터 'cont13'까지의 연속형 특성 목록을 생성합니다.

요약

코드는 주어진 `repl\_list`에 따라 `df\_train`과 `df\_test` 데이터프레임의

특정 범주형 열 값을 치환하고, 치환 결과를 검증하여 적용합니다.

최종적으로 범주형 및 연속형 특성의 목록을 정의하여

이후 분석이나 모델링 작업에 사용할 수 있도록 합니다.

...

처리 과정에 필요하 내용들을 list 형태로 구성합니다.

```
repl_list = [
    ('cat3', {'B': 'C'}, [83634, 147361, 9005]),
    ('cat4', {'A': 'B', 'D': 'B'}, [239397, 603]),
    ('cat6', {'D': 'A', 'E': 'B', 'G': 'C', 'H': 'B', 'I': 'A'}, [234203, 5145, 652]),
    ('cat7', {'A': 'B', 'C': 'B', 'F': 'D', 'I': 'B'}, [4606, 19784, 214027, 1583]),
    ('cat8', {'B': 'G', 'F': 'E'}, [30338, 96743, 2953, 76085, 33881]),
    ('cat9', {'C': 'H', 'D': 'B', 'E': 'L'}, [10678, 2846, 85944, 8320, 19987,
                                             40070, 5501, 16743, 33793, 7819, 3331, 4968])
]
```

반복문 처리 내용들을 수행합니다.

```
for v, d, cnts in repl_list:
    print(v, d, cnts)
    # 치환후 내용을 s_repl에 저장합니다
    s_repl = df_train[v].replace(d)
    # 치환결과를 확인합니다.
    if (s_repl.nunique() != len(cnts)) or \
        ((s_repl.value_counts().sort_index() != cnts).any()):
        print(s_repl.value_counts())
        break
    df_train[v] = s_repl
    df_test[v] = df_test[v].replace(d)
```

```
cat_cols = ['cat{}'.format(i) for i in range(10)]
cont_cols = ['cont{}'.format(i) for i in range(14)]
```

```
cat3 {'B': 'C'} [83634, 147361, 9005]
cat4 {'A': 'B', 'D': 'B'} [239397, 603]
cat6 {'D': 'A', 'E': 'B', 'G': 'C', 'H': 'B', 'I': 'A'} [234203, 5145, 652]
cat7 {'A': 'B', 'C': 'B', 'F': 'D', 'I': 'B'} [4606, 19784, 214027, 1583]
cat8 {'B': 'G', 'F': 'E'} [30338, 96743, 2953, 76085, 33881]
cat9 {'C': 'H', 'D': 'B', 'E': 'L'} [10678, 2846, 85944, 8320, 19987, 40070, 5501, 16743, 33793, 7819, 3331, 4968]
```

```
In [4]: cat_cols
```

```
Out[4]: ['cat0',
        'cat1',
        'cat2',
        'cat3',
        'cat4',
        'cat5',
        'cat6',
        'cat7',
        'cat8',
        'cat9']
```

```
In [5]: cont_cols
```

```
Out[5]: ['cont0',
        'cont1',
        'cont2',
        'cont3',
        'cont4',
        'cont5',
        'cont6',
        'cont7',
        'cont8',
        'cont9',
        'cont10',
        'cont11',
        'cont12',
        'cont13']
```

```
In [6]: '''
```

코드는 `df\_train` 데이터프레임의 `targetA`라는 새로운 타겟 변수를 예측하기 위한 모델을 훈련시키고, 결과를 저장하거나 저장된 결과를 불러오는 작업을 수행합니다. 이 과정에서 시간 절약을 위해 중간 결과를 파일로 저장하고 재사용합니다.

```

# 1. 준비 작업
- `X_all` 변수에 `df_test`의 모든 열 이름을 리스트로 저장합니다.
- `targetA` 변수를 `df_train`에 추가합니다.
  `targetA`는 `target`이 7.45 이하인지 여부를 나타냅니다.
X_all = df_test.columns.tolist()
df_train['targetA'] = df_train['target'] <= 7.45

# 2. 파일 존재 여부 확인
저장된 결과 파일(`targetA_train.csv`)이 존재하는지 확인합니다.
존재하지 않으면 모델 훈련 및 예측 작업을 수행하고, 존재하면 저장된 파일을 불러옵니다.
if not os.path.isfile('targetA_train.csv'):

# 3. 모델 훈련 및 예측 (파일이 없는 경우)
`targetA` 예측 모델을 훈련시키고 예측 결과를 저장합니다.

## 데이터 준비
특정 조건을 만족하는 데이터를 선택하여 새로운 데이터프레임(`df_train_clf`)을 만듭니다.
df_train_clf = df_train.assign(
    prob_A = 1 - norm.cdf(df_train['target'], loc=6.769, scale=0.616),
    prob_B = norm.cdf(df_train['target'], loc=8.123, scale=0.527)
).query('prob_B < 0.01 or prob_A < 0.01').copy()
- `prob_A`와 `prob_B`를 계산하여 새로운 열로 추가합니다.
- `prob_B < 0.01 or prob_A < 0.01` 조건을 만족하는 데이터만 선택합니다.

## 모델 정의 및 훈련
XGBoost 모델을 정의하고 훈련합니다.
clf_xgb = make_pipeline(
    ColumnTransformer([
        ('ohe', OneHotEncoder(handle_unknown='ignore'), cat_cols),
        ('pt', 'passthrough', cont_cols)
    ]),
    xgb.XGBClassifier(
        max_depth=2,
        reg_alpha=0.1,
        reg_lambda=0.1,
        colsample_bytree=0.25,
        n_estimators=500,
        random_state=123,
    )
)
clf_xgb.fit(df_train_clf[X_all], df_train_clf['targetA'])
- `ColumnTransformer`를 사용하여 범주형 열은 원-핫 인코딩하고,

```

```

    연속형 열은 그대로 사용합니다.
- `XGBClassifier`를 설정하고 파이프라인으로 결합하여 훈련합니다.

## 예측 결과 저장
훈련된 모델을 사용하여 `df_train`과 `df_test`에 대한 예측 확률을 계산하고 저장합니다.
df_targetA_train = pd.DataFrame({'targetA_prob': clf_xgb.predict_proba(df_train[X_all])[:, 1]},
                                index=df_train.index)

targetA_prob_cv = cross_val_predict(clf_xgb, df_train_clf[X_all],
                                    df_train_clf['targetA'], cv=5, method='predict_proba')
df_targetA_train.loc[df_targetA_train.index.isin(df_train_clf.index),
                    'targetA_prob'] = targetA_prob_cv[:, 1]
df_targetA_train.to_csv('targetA_train.csv')
df_targetA_test = pd.DataFrame({'targetA_prob': clf_xgb.predict_proba(df_test[X_all])[:, 1]},
                                index=df_test.index)
df_targetA_test.to_csv('targetA_test.csv')
- `df_train`에 대한 예측 확률을 `df_targetA_train`에 저장합니다.
- `cross_val_predict`를 사용하여 교차 검증 예측 결과를 `df_train_clf`에 적용합니다.
- 예측 결과를 CSV 파일로 저장합니다.

# 4. 저장된 파일 불러오기 (파일이 있는 경우)
저장된 예측 결과 파일을 불러옵니다.
else:
    df_targetA_train = pd.read_csv('targetA_train.csv', index_col='id')
    df_targetA_test = pd.read_csv('targetA_test.csv', index_col='id')
- `targetA_train.csv`와 `targetA_test.csv` 파일을 불러와
  `df_targetA_train`와 `df_targetA_test` 데이터프레임에 저장합니다.

### 요약
코드는 `df_train` 데이터프레임의 `targetA` 변수를 예측하기 위해 XGBoost 모델을 훈련시키고,
그 예측 결과를 `df_train`과 `df_test`에 대해 저장하거나 저장된 파일을 불러옵니다.
중간 결과를 파일로 저장하여 다음 실행 시 시간을 절약할 수 있습니다.
...

from sklearn.model_selection import cross_val_predict
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import make_pipeline
from sklearn.compose import ColumnTransformer
from scipy.stats import norm

import os

X_all = df_test.columns.tolist()

```

```

print('X_all = ', X_all)
df_train['targetA'] = df_train['target'] <= 7.45

# 시간이 오래걸리므로 저장해두고, 저장 결과가 없을 시 실행합니다.
# 실제 시험을 고려한 루틴이기 보다는, 이 노트를 리뷰할 때
# 좀 더 시간을 절약할 수 있게 고안된 것입니다.
# 처리과정을 재 실행하려면 생성된 targetA_train.csv를 삭제해야 합니다.
if not os.path.isfile('targetA_train.csv'):
    print('if not os.path.isfile()')
    # 문제 3에서 targetA에 대한 예측 모델을 사용합니다,
    df_train_clf = df_train.assign(
        probb_A = 1 - norm.cdf(df_train['target'], loc=6.769, scale=0.616),
        probb_B = norm.cdf(df_train['target'], loc=8.123, scale=0.527)
    ).query('probb_B < 0.01 or probb_A < 0.01').copy()

    print('df_train_clf.columns = ', df_train_clf.columns)
    print('df_train_clf = ', df_train_clf)

    clf_xgb = make_pipeline(
        ColumnTransformer([
            ('ohe', OneHotEncoder(handle_unknown='ignore'), cat_cols),
            ('pt', 'passthrough', cont_cols)
        ]),
        xgb.XGBClassifier(
            max_depth = 2, # 트리의 최대 깊이 2
            reg_alpha = 0.1, # L1 규제 0.1
            reg_lambda = 0.1, # L2 규제 0.1
            colsample_bytree=0.25, # 트리 당 컬럼 샘플링 비율 0.25
            n_estimators=500, # 트리의 수 500
            random_state=123, # random_state 123
        )
    )

    clf_xgb.fit(df_train_clf[X_all], df_train_clf['targetA'])
    df_targetA_train = pd.DataFrame({'targetA_prob': clf_xgb.predict_proba(df_train[X_all])[:, 1]},
                                    index=df_train.index)

    print('df_targetA_train.columns = ', df_targetA_train.columns)
    print('df_targetA_train = ', df_targetA_train)

    # 여기까지 처리를 통해서도 어느 정도 효과를 얻을 수 있습니다.

```

```

# 두 개의 라인의 처리로 더욱 효과적인 속성이 됩니다.
# cross_val_predict로 쉽게 할 수 있습니다. 교차 검증시 겹외셋에 대한 예측을 결과를 반환합니다.
# train셋에 대한 예측 분포가 아닌 train을 제외한 예측에 대한 분포를 지니도록 합니다.
targetA_prob_cv = cross_val_predict(clf_xgb, df_train_clf[X_all], df_train_clf['targetA'],
                                    cv=5, method='predict_proba')

# df_train_clf에 있는 해당하는 예측은 교차검증의 예측값으로 대체합니다.
# 이 처리를 했을 때와 안 했을 때의 결과 차이를 확인해보시고면, 그 차이를 체감하실 수 있습니다.
df_targetA_train.loc[df_targetA_train.index.isin(df_train_clf.index), 'targetA_prob'] \
    = targetA_prob_cv[:, 1]

df_targetA_train.to_csv('targetA_train.csv')
df_targetA_test = pd.DataFrame({'targetA_prob': clf_xgb.predict_proba(df_test[X_all])[:, 1]},
                              index=df_test.index)
df_targetA_test.to_csv('targetA_test.csv')
else:
    print('else')
    # 저장된 결과를 불러옵니다.
    df_targetA_train = pd.read_csv('targetA_train.csv', index_col='id')
    df_targetA_test = pd.read_csv('targetA_test.csv', index_col='id')

```

```

X_all = ['cat0', 'cat1', 'cat2', 'cat3', 'cat4', 'cat5', 'cat6', 'cat7', 'cat8', 'cat9', 'cont0', 'cont1', 'cont2', 'cont3', 'cont4', 'cont5', 'cont6', 'cont7', 'cont8', 'cont9', 'cont10', 'cont11', 'cont12', 'cont13']
else

```

```

In [7]: df_train['targetA_prob'] = df_targetA_train['targetA_prob']
df_test['targetA_prob'] = df_targetA_test['targetA_prob']

```

```

In [8]: '''
코드는 `df_train`과 `df_test` 데이터프레임에 연속형 변수들의 새로운 파생 변수를
추가하는 작업을 수행합니다. 구체적으로, 각 연속형 변수의 값에 대한 구간을 나누고,
그 구간별로 `target` 변수의 평균을 계산하여 새로운 변수를 생성합니다.

# 1. 새로운 파생 변수 추가
우선, 이전 단계에서 생성된 `targetA_prob` 변수를 `df_train`과 `df_test`에 추가합니다.

# 2. 구간 설정
0부터 1까지 0.01 간격의 값을 포함하는 리스트 `q`를 생성합니다.
이는 각 연속형 변수를 100개의 구간으로 나누기 위한 것입니다.

# 3. 각 연속형 변수에 대해 파생 변수 생성
`cont0`부터 `cont13`까지의 연속형 변수에 대해 새로운 파생 변수를 생성합니다.
반복문을 통해 각 연속형 변수에 대해 다음 작업을 수행합니다.
'''

```


3.1. 구간 계산

각 변수의 값에 대해 지정된 분위수(`q`)에 해당하는 값을 계산합니다.
`q\_val`은 각 구간의 경계값을 나타냅니다.

3.2. 경계값 조정

경계값의 첫 번째(`0%`)와 마지막(`100%`) 값을 각각 `-np.inf`와
`np.inf`로 설정하여 모든 값을 포함할 수 있도록 합니다.

3.3. 구간 나누기

`pd.cut`을 사용하여 각 변수의 값을 `q\_val`에 따라 구간으로 나눕니다.
`q\_range`는 각 값이 속하는 구간을 나타냅니다.

3.4. 구간별 평균 계산

각 구간에 속하는 `target` 변수의 평균을 계산하여 `q\_mean`에 저장합니다.

3.5. 새로운 파생 변수 생성

각 값의 구간을 `q\_mean`의 값으로 매핑하여 새로운 파생 변수를 생성합니다.
이를 `df\_train`과 `df\_test`에 추가합니다.

```
- `df_train[col + '_q']`:
    `df_train`의 각 값을 구간에 매핑한 후, 그 구간의 `target` 평균 값으로 대체합니다.
- `df_test[col + '_q']`:
    `df_test`의 각 값을 `df_train`에서 계산된 구간에 매핑한 후,
    그 구간의 `target` 평균 값으로 대체합니다.
```

요약

코드는 `df\_train`과 `df\_test`의 각 연속형 변수에 대해 구간별로
`target`의 평균 값을 새로운 파생 변수로 추가합니다.
이를 통해 각 연속형 변수를 구간화하고, 그 구간의 `target` 평균 값을
변수로 사용하여 모델의 성능을 향상시킬 수 있습니다.

```
q = [i for i in np.arange(0, 1.01, 0.01)]
print('q = ', q)
print('-----')
# 나머지 변수에 대해서도 해당 파생 변수를 만들어 줍니다.
for i in range(0, 14):
    col = 'cont{}'.format(i)
    q_val = df_train[col].quantile(q)
    q_val.iloc[[0, -1]] = [-np.inf, np.inf]
    print('q_val = \n', q_val)
    print('-----')
```

```
q_range = pd.cut(df_train[col], bins=q_val)
print('q_range = \n', q_range)
print('-----')
q_mean = df_train.groupby(q_range)['target'].mean()
print('q_mean = \n', q_mean)
print('-----')
df_train[col + '_q'] = q_range.map(q_mean).astype('float')
print('df_train[col + _q] = \n', df_train[col + '_q'])
print('-----')
df_test[col + '_q'] = pd.cut(df_test[col], bins=q_val).map(q_mean).astype('float')
print('df_test[col + _q] = \n', df_test[col + '_q'])
print('-----')
```

```
q = [0.0, 0.01, 0.02, 0.03, 0.04, 0.05, 0.06, 0.07, 0.08, 0.09, 0.1, 0.11, 0.12, 0.13, 0.14, 0.15, 0.16, 0.17, 0.18, 0.19, 0.2,
0.21, 0.22, 0.23, 0.24, 0.25, 0.26, 0.27, 0.28, 0.29, 0.3, 0.31, 0.32, 0.33, 0.34, 0.35000000000000003, 0.36, 0.37, 0.38, 0.39,
0.4, 0.41000000000000003, 0.42, 0.43, 0.44, 0.45, 0.46, 0.47000000000000003, 0.48, 0.49, 0.5, 0.51, 0.52, 0.53, 0.54, 0.55, 0.5
6, 0.57000000000000001, 0.58, 0.59, 0.6, 0.61, 0.62, 0.63, 0.64, 0.65, 0.66, 0.67, 0.68, 0.69000000000000001, 0.70000000000000001,
0.71, 0.72, 0.73, 0.74, 0.75, 0.76, 0.77, 0.78, 0.79, 0.8, 0.81, 0.82000000000000001, 0.83000000000000001, 0.84, 0.85, 0.86, 0.87,
0.88, 0.89, 0.9, 0.91, 0.92, 0.93, 0.94000000000000001, 0.95000000000000001, 0.96, 0.97, 0.98, 0.99, 1.0]
```

```
-----
q_val =
0.00      -inf
0.01      0.110730
0.02      0.174260
0.03      0.219630
0.04      0.239460
...
0.96      0.944610
0.97      0.956960
0.98      0.970251
0.99      0.986900
1.00      inf
```

```
Name: cont0, Length: 101, dtype: float64
```

```
-----
q_range =
id
267387    (0.648, 0.655]
410037    (0.364, 0.371]
139373    (0.477, 0.48]
113765    (0.428, 0.437]
179915    (0.477, 0.48]
...
320975    (0.352, 0.358]
29591     (0.536, 0.544]
46717     (0.871, 0.885]
463191    (0.346, 0.352]
415856    (0.871, 0.885]
```

```
Name: cont0, Length: 240000, dtype: category
```

```
Categories (100, interval[float64]): [(-inf, 0.111] < (0.111, 0.174] < (0.174, 0.22] < (0.22, 0.239] ... (0.945, 0.957] < (0.95
7, 0.97] < (0.97, 0.987] < (0.987, inf]]
```

```
-----
q_mean =
cont0
(-inf, 0.111]      7.757739
(0.111, 0.174]     7.659877
```

```
(0.174, 0.22]      7.643175
(0.22, 0.239]     7.653696
(0.239, 0.252]     7.619602
...
(0.933, 0.945]     7.501515
(0.945, 0.957]     7.422674
(0.957, 0.97]      7.368499
(0.97, 0.987]      7.331179
(0.987, inf]       7.260526
```

Name: target, Length: 100, dtype: float64

```
-----
df_train[col + _q] =
```

```
  id
267387    7.309811
410037    7.358717
139373    7.481492
113765    7.566321
179915    7.481492
```

```
...
320975    7.404878
29591     7.346581
46717     7.602596
463191    7.419785
415856    7.602596
```

Name: cont0_q, Length: 240000, dtype: float64

```
-----
df_test[col + _q] =
```

```
  id
464589    7.327784
39160     7.552257
55733     7.472187
97702     7.400330
337310    7.477633
```

```
...
201019    7.297783
336931    7.566321
289650    7.323047
177173    7.475328
199153    7.381399
```

Name: cont0_q, Length: 60000, dtype: float64

```
-----
q_val =
```

```

0.00      -inf
0.01      0.00597
0.02      0.02052
0.03      0.03144
0.04      0.04054

```

```

...
0.96      0.77948
0.97      0.78393
0.98      0.78988
0.99      0.79851

```

```

1.00      inf

```

Name: cont1, Length: 101, dtype: float64

q_range =

```

id
267387    (0.5562, 0.6152]
410037    (0.6783, 0.6828]
139373    (0.6371, 0.6414]
113765    (0.7757, 0.7795]
179915    (0.6414, 0.6456]

```

```

...
320975    (0.7613, 0.7637]
29591     (0.5522, 0.5528]
46717     (0.6878, 0.6938]
463191    (0.6632, 0.6667]
415856    (0.7377, 0.741]

```

Name: cont1, Length: 24000, dtype: category

Categories (100, interval[float64]): [(-inf, 0.00597] < (0.00597, 0.02052] < (0.02052, 0.03144] < (0.03144, 0.04054] ... (0.7795, 0.7839] < (0.7839, 0.7899] < (0.7899, 0.7985] < (0.7985, inf]]

q_mean =

```

cont1
(-inf, 0.00597]      7.316447
(0.00597, 0.02052]   7.292605
(0.02052, 0.03144]   7.300449
(0.03144, 0.04054]   7.344934
(0.04054, 0.04918]   7.387014

```

```

...
(0.7757, 0.7795]     7.447119
(0.7795, 0.7839]     7.508745
(0.7839, 0.7899]     7.481619
(0.7899, 0.7985]     7.525803

```

```
(0.7985, inf]      7.551942
Name: target, Length: 100, dtype: float64
```

```
-----
df_train[col + _q] =
```

```
id
267387    7.504628
410037    7.501029
139373    7.448435
113765    7.447119
179915    7.451060
```

```
...
320975    7.410879
29591     7.506200
46717     7.552151
463191    7.495144
415856    7.377689
```

```
Name: cont1_q, Length: 240000, dtype: float64
```

```
-----
df_test[col + _q] =
```

```
id
464589    7.448435
39160     7.525803
55733     7.372920
97702     7.398788
337310    7.431725
```

```
...
201019    7.533076
336931    7.456210
289650    7.482514
177173    7.481619
199153    7.530238
```

```
Name: cont1_q, Length: 60000, dtype: float64
```

```
-----
q_val =
```

```
0.00      -inf
0.01     0.026690
0.02     0.069979
0.03     0.102989
0.04     0.117280
```

```
...
0.96     0.821882
0.97     0.898701
```

0.98 0.931580

0.99 0.940460

1.00 inf

Name: cont2, Length: 101, dtype: float64

q_range =

id

267387 (0.246, 0.261]

410037 (0.279, 0.301]

139373 (0.397, 0.403]

113765 (0.548, 0.549]

179915 (0.365, 0.376]

...

320975 (0.339, 0.345]

29591 (0.323, 0.327]

46717 (0.188, 0.196]

463191 (0.432, 0.438]

415856 (0.174, 0.181]

Name: cont2, Length: 240000, dtype: category

Categories (100, interval[float64]): [(-inf, 0.0267] < (0.0267, 0.07] < (0.07, 0.103] < (0.103, 0.117] ... (0.822, 0.899] < (0.899, 0.932] < (0.932, 0.94] < (0.94, inf]]

q_mean =

cont2

(-inf, 0.0267] 7.350697

(0.0267, 0.07] 7.375655

(0.07, 0.103] 7.399755

(0.103, 0.117] 7.422285

(0.117, 0.128] 7.431159

...

(0.785, 0.822] 7.494986

(0.822, 0.899] 7.476301

(0.899, 0.932] 7.420228

(0.932, 0.94] 7.460624

(0.94, inf] 7.476024

Name: target, Length: 100, dtype: float64

df_train[col + _q] =

id

267387 7.558684

410037 7.476887

139373 7.402963

```

113765    7.433759
179915    7.379472
...
320975    7.484996
29591     7.469575
46717     7.473212
463191    7.437155
415856    7.520621

```

Name: cont2_q, Length: 240000, dtype: float64

```
df_test[col + _q] =
```

```

  id
464589    7.454337
39160     7.484996
55733     7.431159
97702     7.483363
337310    7.494986

```

```

...
201019    7.477116
336931    7.460624
289650    7.471470
177173    7.456613
199153    7.481695

```

Name: cont2_q, Length: 60000, dtype: float64

```
q_val =
```

```

0.00      -inf
0.01     0.15465
0.02     0.15989
0.03     0.16368
0.04     0.16695

```

```

...
0.96     0.83209
0.97     0.84223
0.98     0.85542
0.99     0.87484

```

```
1.00      inf
```

Name: cont3, Length: 101, dtype: float64

```
q_range =
```

```

  id
267387    (0.279, 0.292]

```



```

410037    (0.201, 0.203]
139373    (0.241, 0.248]
113765    (0.217, 0.22]
179915    (0.21, 0.212]

...
320975    (0.164, 0.167]
29591     (0.343, 0.347]
46717     (0.786, 0.791]
463191    (0.18, 0.182]
415856    (0.236, 0.241]
Name: cont3, Length: 240000, dtype: category
Categories (100, interval[float64]): [(-inf, 0.155] < (0.155, 0.16] < (0.16, 0.164] < (0.164, 0.167] ... (0.832, 0.842] < (0.842, 0.855] < (0.855, 0.875] < (0.875, inf]]
-----
q_mean =
  cont3
(-inf, 0.155]    7.325525
(0.155, 0.16]    7.354220
(0.16, 0.164]    7.382812
(0.164, 0.167]   7.384493
(0.167, 0.17]    7.388060

...
(0.823, 0.832]   7.502934
(0.832, 0.842]   7.524561
(0.842, 0.855]   7.553160
(0.855, 0.875]   7.539267
(0.875, inf]     7.561332
Name: target, Length: 100, dtype: float64
-----
df_train[col + _q] =
  id
267387    7.579267
410037    7.499973
139373    7.547704
113765    7.552643
179915    7.496568

...
320975    7.384493
29591     7.417699
46717     7.451499
463191    7.448513
415856    7.563064

```

Name: cont3_q, Length: 240000, dtype: float64

df_test[col + _q] =

id	
464589	7.383482
39160	7.452229
55733	7.553160
97702	7.400624
337310	7.388060

...

201019	7.443519
336931	7.417983
289650	7.504945
177173	7.442591
199153	7.430290

Name: cont3_q, Length: 60000, dtype: float64

q_val =

0.00	-inf
0.01	0.24391
0.02	0.25793
0.03	0.26869
0.04	0.27624

...

0.96	0.84006
0.97	0.85294
0.98	0.86731
0.99	0.88657
1.00	inf

Name: cont4, Length: 101, dtype: float64

q_range =

id	
267387	(0.8143, 0.8269]
410037	(0.456, 0.465]
139373	(0.2687, 0.2762]
113765	(0.2794, 0.2795]
179915	(0.4113, 0.4134]

...

320975	(0.2801, 0.2803]
29591	(0.5248, 0.5339]
46717	(0.5166, 0.5248]

```

463191      (0.328, 0.338]
415856      (0.4202, 0.4225]
Name: cont4, Length: 240000, dtype: category
Categories (100, interval[float64]): [(-inf, 0.2439] < (0.2439, 0.2579] < (0.2579, 0.2687] < (0.2687, 0.2762] ... (0.8401, 0.852
9] < (0.8529, 0.8673] < (0.8673, 0.8866] < (0.8866, inf]]

```

```

-----
q_mean =
  cont4
(-inf, 0.2439]      7.468171
(0.2439, 0.2579]    7.471124
(0.2579, 0.2687]    7.498841
(0.2687, 0.2762]    7.477034
(0.2762, 0.2769]    7.501066
...
(0.8269, 0.8401]    7.431376
(0.8401, 0.8529]    7.472325
(0.8529, 0.8673]    7.456921
(0.8673, 0.8866]    7.420813
(0.8866, inf]       7.386459
Name: target, Length: 100, dtype: float64

```

```

-----
df_train[col + _q] =
  id
267387      7.447799
410037      7.495858
139373      7.477034
113765      7.475138
179915      7.413782
...
320975      7.462526
29591       7.513261
46717       7.526339
463191      7.466972
415856      7.394654
Name: cont4_q, Length: 240000, dtype: float64

```

```

-----
df_test[col + _q] =
  id
464589      7.440512
39160       7.475138
55733       7.501755
97702       7.501857

```

```
337310    7.388361
```

```
...
```

```
201019    7.535606
```

```
336931    7.459748
```

```
289650    7.485857
```

```
177173    7.427872
```

```
199153    7.471124
```

```
Name: cont4_q, Length: 60000, dtype: float64
```

```
-----
```

```
q_val =
```

```
0.00      -inf
```

```
0.01      0.17796
```

```
0.02      0.19019
```

```
0.03      0.19909
```

```
0.04      0.20562
```

```
...
```

```
0.96      0.93246
```

```
0.97      0.94377
```

```
0.98      0.95815
```

```
0.99      0.97979
```

```
1.00      inf
```

```
Name: cont5, Length: 101, dtype: float64
```

```
-----
```

```
q_range =
```

```
id
```

```
267387    (0.221, 0.226]
```

```
410037    (0.432, 0.437]
```

```
139373    (0.362, 0.365]
```

```
113765    (0.318, 0.33]
```

```
179915    (0.199, 0.206]
```

```
...
```

```
320975    (0.609, 0.622]
```

```
29591     (0.836, 0.85]
```

```
46717     (0.898, 0.906]
```

```
463191    (0.247, 0.251]
```

```
415856    (0.923, 0.932]
```

```
Name: cont5, Length: 240000, dtype: category
```

```
Categories (100, interval[float64]): [(-inf, 0.178] < (0.178, 0.19] < (0.19, 0.199] < (0.199, 0.206] ... (0.932, 0.944] < (0.944, 0.958] < (0.958, 0.98] < (0.98, inf]]
```

```
-----
```

```
q_mean =
```

```
cont5
```

```
(-inf, 0.178]      7.746700
(0.178, 0.19]      7.654040
(0.19, 0.199]      7.601649
(0.199, 0.206]      7.621485
(0.206, 0.211]      7.547932
```

...

```
(0.923, 0.932]      7.483913
(0.932, 0.944]      7.475092
(0.944, 0.958]      7.468161
(0.958, 0.98]       7.446681
(0.98, inf]         7.419356
```

Name: target, Length: 100, dtype: float64

```
-----
df_train[col + _q] =
```

```
id
267387      7.553739
410037      7.489556
139373      7.475350
113765      7.471483
179915      7.621485
```

...

```
320975      7.351768
29591       7.561538
46717       7.495275
463191      7.467019
415856      7.483913
```

Name: cont5_q, Length: 240000, dtype: float64

```
-----
df_test[col + _q] =
```

```
id
464589      7.417182
39160       7.409817
55733       7.361008
97702       7.475092
337310      7.394746
```

...

```
201019      7.519039
336931      7.460049
289650      7.519039
177173      7.469504
199153      7.553739
```

Name: cont5_q, Length: 60000, dtype: float64

```

-----
q_val =
  0.00      -inf
  0.01      0.198400
  0.02      0.217080
  0.03      0.229310
  0.04      0.239260
  ...
  0.96      0.889860
  0.97      0.912311
  0.98      0.938330
  0.99      0.972870
  1.00      inf
Name: cont6, Length: 101, dtype: float64
-----

```

```

-----
q_range =
  id
267387      (0.686, 0.698]
410037      (0.938, 0.973]
139373      (0.41, 0.414]
113765      (0.421, 0.425]
179915      (0.372, 0.375]
  ...
320975      (0.407, 0.41]
29591       (0.744, 0.755]
46717       (0.473, 0.482]
463191      (0.198, 0.217]
415856      (0.828, 0.848]
Name: cont6, Length: 240000, dtype: category
Categories (100, interval[float64]): [(-inf, 0.198] < (0.198, 0.217] < (0.217, 0.229] < (0.229, 0.239] ... (0.89, 0.912] < (0.912, 0.938] < (0.938, 0.973] < (0.973, inf]]
-----

```

```

-----
q_mean =
  cont6
(-inf, 0.198]      7.209794
(0.198, 0.217]     7.341439
(0.217, 0.229]     7.339143
(0.229, 0.239]     7.375317
(0.239, 0.248]     7.399317
  ...
(0.869, 0.89]      7.478882
(0.89, 0.912]      7.475926
-----

```

```
(0.912, 0.938]    7.532244
(0.938, 0.973]    7.552695
(0.973, inf]      7.570542
Name: target, Length: 100, dtype: float64
```

```
-----
df_train[col + _q] =
```

```
id
267387    7.361472
410037    7.552695
139373    7.535310
113765    7.506197
179915    7.464368
...
320975    7.533125
29591     7.477164
46717     7.422131
463191    7.341439
415856    7.493042
```

```
Name: cont6_q, Length: 240000, dtype: float64
```

```
-----
df_test[col + _q] =
```

```
id
464589    7.483786
39160     7.427212
55733     7.433168
97702     7.464368
337310    7.506466
...
201019    7.477164
336931    7.533125
289650    7.441826
177173    7.508998
199153    7.425882
```

```
Name: cont6_q, Length: 60000, dtype: float64
```

```
-----
q_val =
```

```
0.00      -inf
0.01     0.22678
0.02     0.23123
0.03     0.23452
0.04     0.23725
...
```

```

0.96    0.87664
0.97    0.89622
0.98    0.91722
0.99    0.94227
1.00      inf

```

Name: cont7, Length: 101, dtype: float64

q_range =

```

id
267387    (0.308, 0.311]
410037    (0.624, 0.632]
139373    (0.27, 0.272]
113765    (0.283, 0.285]
179915    (0.46, 0.47]

```

...

```

320975    (0.624, 0.632]
29591     (0.824, 0.854]
46717     (0.78, 0.798]
463191    (-inf, 0.227]
415856    (0.48, 0.489]

```

Name: cont7, Length: 240000, dtype: category

Categories (100, interval[float64]): [(-inf, 0.227] < (0.227, 0.231] < (0.231, 0.235] < (0.235, 0.237] ... (0.877, 0.896] < (0.896, 0.917] < (0.917, 0.942] < (0.942, inf]]

q_mean =

```

cont7
(-inf, 0.227]    7.535289
(0.227, 0.231]   7.492057
(0.231, 0.235]   7.472763
(0.235, 0.237]   7.473850
(0.237, 0.24]    7.441738

```

...

```

(0.854, 0.877]   7.544543
(0.877, 0.896]   7.569541
(0.896, 0.917]   7.547892
(0.917, 0.942]   7.505851
(0.942, inf]     7.463586

```

Name: target, Length: 100, dtype: float64

df_train[col + _q] =

```

id
267387    7.425451

```



```
410037    7.470544
139373    7.427466
113765    7.443850
179915    7.526495
```

```
...
320975    7.470544
29591     7.573120
46717     7.512303
463191    7.535289
415856    7.487008
```

Name: cont7_q, Length: 240000, dtype: float64

```
df_test[col + _q] =
```

```
id
464589    7.442454
39160     7.444641
55733     7.569541
97702     7.463586
337310    7.423142
```

```
...
201019    7.487008
336931    7.469328
289650    7.461077
177173    7.427863
199153    7.472763
```

Name: cont7_q, Length: 60000, dtype: float64

```
q_val =
```

```
0.00      -inf
0.01      0.07230
0.02      0.09466
0.03      0.11158
0.04      0.12689
```

```
...
0.96      0.91578
0.97      0.92472
0.98      0.93592
0.99      0.95134
```

```
1.00      inf
```

Name: cont8, Length: 101, dtype: float64

```
q_range =
```

```

id
267387      (0.47, 0.474]
410037      (0.399, 0.404]
139373      (0.399, 0.404]
113765      (0.464, 0.467]
179915      (0.741, 0.788]
...
320975      (0.312, 0.316]
29591       (0.464, 0.467]
46717       (0.874, 0.881]
463191      (-inf, 0.0723]
415856      (0.901, 0.908]
Name: cont8, Length: 240000, dtype: category
Categories (100, interval[float64]): [(-inf, 0.0723] < (0.0723, 0.0947] < (0.0947, 0.112] < (0.112, 0.127] ... (0.916, 0.925] <
(0.925, 0.936] < (0.936, 0.951] < (0.951, inf]]
-----

q_mean =
cont8
(-inf, 0.0723]      7.157330
(0.0723, 0.0947]    7.253841
(0.0947, 0.112]     7.312009
(0.112, 0.127]      7.291105
(0.127, 0.141]      7.352542
...
(0.908, 0.916]      7.550353
(0.916, 0.925]      7.601496
(0.925, 0.936]      7.622044
(0.936, 0.951]      7.684482
(0.951, inf]         7.794472
Name: target, Length: 100, dtype: float64
-----

df_train[col + _q] =
id
267387      7.422161
410037      7.429530
139373      7.429530
113765      7.476342
179915      7.509317
...
320975      7.606867
29591       7.476342
46717       7.392892

```

```

463191    7.157330
415856    7.520757
Name: cont8_q, Length: 240000, dtype: float64

```

```

-----
df_test[col + _q] =

```

```

    id
464589    7.390354
39160     7.424690
55733     7.402902
97702     7.794472
337310    7.408953

```

```

    ...
201019    7.201716
336931    7.601494
289650    7.557812
177173    7.606867
199153    7.454519

```

```

Name: cont8_q, Length: 60000, dtype: float64

```

```

-----
q_val =

```

```

    0.00    -inf
0.01    0.160750
0.02    0.184550
0.03    0.202349
0.04    0.218070

```

```

    ...
0.96    0.838000
0.97    0.843480
0.98    0.850490
0.99    0.873830

```

```

    1.00     inf

```

```

Name: cont9, Length: 101, dtype: float64

```

```

-----
q_range =

```

```

    id
267387    (0.53, 0.536]
410037    (0.489, 0.497]
139373    (0.53, 0.536]
113765    (0.383, 0.386]
179915    (0.554, 0.561]

```

```

    ...
320975    (0.436, 0.441]

```

```

29591      (0.834, 0.838]
46717      (0.635, 0.648]
463191     (0.612, 0.624]
415856     (0.815, 0.818]
Name: cont9, Length: 240000, dtype: category
Categories (100, interval[float64]): [(-inf, 0.161] < (0.161, 0.185] < (0.185, 0.202] < (0.202, 0.218] ... (0.838, 0.843] < (0.8
43, 0.85] < (0.85, 0.874] < (0.874, inf]]
-----
q_mean =
  cont9
(-inf, 0.161]      7.298128
(0.161, 0.185]     7.417046
(0.185, 0.202]     7.458164
(0.202, 0.218]     7.453086
(0.218, 0.232]     7.520364
...
(0.834, 0.838]     7.577215
(0.838, 0.843]     7.567094
(0.843, 0.85]      7.614762
(0.85, 0.874]      7.561821
(0.874, inf]       7.504509
Name: target, Length: 100, dtype: float64
-----
df_train[col + _q] =
  id
267387      7.540963
410037      7.465038
139373      7.540963
113765      7.449267
179915      7.535321
...
320975      7.342920
29591       7.577215
46717       7.371871
463191      7.367900
415856      7.522725
Name: cont9_q, Length: 240000, dtype: float64
-----
df_test[col + _q] =
  id
464589      7.490189
39160       7.459231

```

```
55733      7.309593
97702      7.571929
337310     7.486320
```

```
...
201019     7.276681
336931     7.551569
289650     7.309593
177173     7.276681
199153     7.472503
```

Name: cont9_q, Length: 60000, dtype: float64

q_val =

```
0.00      -inf
0.01      0.14080
0.02      0.15931
0.03      0.17286
0.04      0.18451
```

```
...
0.96      0.85915
0.97      0.88238
0.98      0.90794
0.99      0.94084
1.00      inf
```

Name: cont10, Length: 101, dtype: float64

q_range =

```
id
267387      (0.941, inf]
410037      (0.56, 0.565]
139373      (0.355, 0.36]
113765      (0.379, 0.385]
179915      (0.308, 0.315]
```

```
...
320975      (0.403, 0.411]
29591       (0.716, 0.72]
46717       (0.587, 0.591]
463191      (0.618, 0.623]
415856      (0.575, 0.579]
```

Name: cont10, Length: 240000, dtype: category

Categories (100, interval[float64]): [(-inf, 0.141] < (0.141, 0.159] < (0.159, 0.173] < (0.173, 0.185] ... (0.859, 0.882] < (0.882, 0.908] < (0.908, 0.941] < (0.941, inf]]

```

q_mean =
  cont10
(-inf, 0.141]      7.597994
(0.141, 0.159]     7.508948
(0.159, 0.173]     7.541271
(0.173, 0.185]     7.518211
(0.185, 0.195]     7.510061
...
(0.838, 0.859]     7.483050
(0.859, 0.882]     7.563248
(0.882, 0.908]     7.554312
(0.908, 0.941]     7.500854
(0.941, inf]       7.504011
Name: target, Length: 100, dtype: float64

```

```

-----
df_train[col + _q] =
  id
267387      7.504011
410037      7.420761
139373      7.459387
113765      7.487536
179915      7.444311
...
320975      7.405092
29591       7.423979
46717       7.393477
463191      7.369908
415856      7.449775
Name: cont10_q, Length: 240000, dtype: float64

```

```

-----
df_test[col + _q] =
  id
464589      7.435756
39160       7.500854
55733       7.545148
97702       7.507000
337310      7.432532
...
201019      7.387827
336931      7.421740
289650      7.476759
177173      7.452681

```

199153 7.597994

Name: cont10_q, Length: 60000, dtype: float64

q_val =

0.00 -inf

0.01 0.14342

0.02 0.16070

0.03 0.17288

0.04 0.18325

...

0.96 0.87987

0.97 0.89364

0.98 0.91206

0.99 0.94186

1.00 inf

Name: cont11, Length: 101, dtype: float64

q_range =

id

267387 (0.592, 0.605]

410037 (0.393, 0.401]

139373 (0.328, 0.333]

113765 (0.688, 0.691]

179915 (0.42, 0.434]

...

320975 (0.697, 0.7]

29591 (0.694, 0.697]

46717 (0.777, 0.784]

463191 (0.251, 0.257]

415856 (0.451, 0.469]

Name: cont11, Length: 240000, dtype: category

Categories (100, interval[float64]): [(-inf, 0.143] < (0.143, 0.161] < (0.161, 0.173] < (0.173, 0.183] ... (0.88, 0.894] < (0.894, 0.912] < (0.912, 0.942] < (0.942, inf]]

q_mean =

cont11

(-inf, 0.143] 7.287957

(0.143, 0.161] 7.320170

(0.161, 0.173] 7.398482

(0.173, 0.183] 7.408767

(0.183, 0.192] 7.394984

...

```
(0.868, 0.88]      7.607627
(0.88, 0.894]      7.610827
(0.894, 0.912]      7.561680
(0.912, 0.942]      7.557726
(0.942, inf]        7.613859
Name: target, Length: 100, dtype: float64
```

```
-----
df_train[col + _q] =
  id
```

```
267387    7.321437
410037    7.567407
139373    7.483656
113765    7.329234
179915    7.474959
```

```
...
320975    7.390826
29591     7.382039
46717     7.490759
463191    7.508202
415856    7.356940
```

```
Name: cont11_q, Length: 240000, dtype: float64
```

```
-----
df_test[col + _q] =
  id
```

```
464589    7.519634
39160     7.615007
55733     7.321437
97702     7.560002
337310    7.405645
```

```
...
201019    7.420012
336931    7.533831
289650    7.310597
177173    7.382039
199153    7.498218
```

```
Name: cont11_q, Length: 60000, dtype: float64
```

```
-----
q_val =
```

```
0.00      -inf
0.01     0.23318
0.02     0.25306
0.03     0.26331
```


0.04 0.27076

...

0.96 0.88776

0.97 0.89477

0.98 0.90315

0.99 0.91429

1.00 inf

Name: cont12, Length: 101, dtype: float64

q_range =

id

267387 (0.367, 0.37]

410037 (0.362, 0.364]

139373 (0.332, 0.334]

113765 (0.347, 0.349]

179915 (0.323, 0.325]

...

320975 (0.344, 0.346]

29591 (0.329, 0.33]

46717 (0.863, 0.869]

463191 (0.332, 0.334]

415856 (0.914, inf]

Name: cont12, Length: 240000, dtype: category

Categories (100, interval[float64]): [(-inf, 0.233] < (0.233, 0.253] < (0.253, 0.263] < (0.263, 0.271] ... (0.888, 0.895] < (0.895, 0.903] < (0.903, 0.914] < (0.914, inf]]

q_mean =

cont12

(-inf, 0.233] 7.426257

(0.233, 0.253] 7.448829

(0.253, 0.263] 7.447384

(0.263, 0.271] 7.474044

(0.271, 0.277] 7.436660

...

(0.881, 0.888] 7.527291

(0.888, 0.895] 7.499579

(0.895, 0.903] 7.552917

(0.903, 0.914] 7.559799

(0.914, inf] 7.549361

Name: target, Length: 100, dtype: float64

df_train[col + _q] =

```

id
267387    7.448017
410037    7.444624
139373    7.481380
113765    7.474225
179915    7.456458
...
320975    7.405900
29591     7.467025
46717     7.493169
463191    7.481380
415856    7.549361
Name: cont12_q, Length: 240000, dtype: float64

```

```

-----
df_test[col + _q] =
  id
464589    7.465130
39160     7.476420
55733     7.493169
97702     7.505730
337310    7.331958
...
201019    7.527291
336931    7.446289
289650    7.325204
177173    7.446289
199153    7.436931
Name: cont12_q, Length: 60000, dtype: float64

```

```

-----
q_val =
  0.00    -inf
  0.01    0.18407
  0.02    0.19180
  0.03    0.19744
  0.04    0.20244
...
  0.96    0.82040
  0.97    0.82444
  0.98    0.82946
  0.99    0.83628
  1.00     inf
Name: cont13, Length: 101, dtype: float64

```

```

-----
q_range =
  id
267387    (0.421, 0.434]
410037    (0.715, 0.719]
139373    (0.824, 0.829]
113765    (0.731, 0.735]
179915    (-inf, 0.184]
...
320975    (0.675, 0.681]
29591     (0.284, 0.286]
46717     (0.744, 0.749]
463191    (0.282, 0.284]
415856    (0.254, 0.262]
Name: cont13, Length: 240000, dtype: category
Categories (100, interval[float64]): [(-inf, 0.184] < (0.184, 0.192] < (0.192, 0.197] < (0.197, 0.202] ... (0.82, 0.824] < (0.824, 0.829] < (0.829, 0.836] < (0.836, inf]]
-----

```

```

q_mean =
  cont13
(-inf, 0.184]    7.358709
(0.184, 0.192]   7.388248
(0.192, 0.197]   7.362520
(0.197, 0.202]   7.386443
(0.202, 0.207]   7.380244
...
(0.817, 0.82]    7.323905
(0.82, 0.824]    7.353536
(0.824, 0.829]   7.347986
(0.829, 0.836]   7.363639
(0.836, inf]     7.426940
Name: target, Length: 100, dtype: float64
-----

```

```

df_train[col + _q] =
  id
267387    7.474239
410037    7.585489
139373    7.347986
113765    7.598602
179915    7.358709
...
320975    7.544139

```

```

29591      7.460992
46717      7.611174
463191     7.435650
415856     7.442899
Name: cont13_q, Length: 240000, dtype: float64
-----

```

```

df_test[col + _q] =
  id
464589     7.598602
39160      7.443049
55733      7.585489
97702      7.332872
337310     7.439257
...
201019     7.440075
336931     7.269783
289650     7.576272
177173     7.521038
199153     7.353536
Name: cont13_q, Length: 60000, dtype: float64
-----

```

```
In [9]: s_hist = []
```

```
In [10]: '''
`ShuffleSplit`은 사이킷런(sklearn)의 `model_selection` 모듈에 있는 클래스 중 하나로,
데이터셋을 섞어서 훈련 세트와 테스트 세트로 나누는 데 사용됩니다.
`ShuffleSplit`은 KFold와 유사하게 교차 검증(cross-validation)을 수행하지만,
샘플링과 분할이 독립적으로 이루어지며 데이터가 무작위로 섞인 상태로 유지됩니다.

### 1. `ShuffleSplit`의 주요 인자 (Parameters)
- n_splits: `int`, default=10
  데이터셋을 분할하는 횟수(즉, 반복할 횟수)를 지정합니다. 기본값은 10입니다.
- test_size: `float`, `int`, or `None`, default=0.1
  테스트 세트의 크기를 설정합니다.
  - `float`일 경우, 전체 데이터셋에 대한 비율로 설정됩니다 (예: `test_size=0.2`는 20%를 테스트 세트로 사용).
  - `int`일 경우, 테스트 세트의 샘플 수를 명시적으로 설정합니다 (예: `test_size=10`은 테스트 세트에 10개의 샘플을 사용).
  - `None`일 경우, `train_size`와 `n_samples`를 사용하여 크기가 자동으로 결정됩니다.
- train_size: `float`, `int`, or `None`, default=None
  훈련 세트의 크기를 설정합니다.
  - `float`일 경우, 전체 데이터셋에 대한 비율로 설정됩니다 (예: `train_size=0.8`는 80%를 훈련 세트로 사용).
  - `int`일 경우, 훈련 세트의 샘플 수를 명시적으로 설정합니다.
'''
```

```

- `None`일 경우, `test_size`로부터 크기가 자동으로 결정됩니다.
- random_state: `int`, `RandomState` instance or `None`, default=None
  난수 생성에 사용되는 시드를 설정합니다. 이 값을 설정하면 같은 분할을 재현할 수 있습니다.

### 2. 리턴값 (Returns)
- `split()` 메소드는 교차 검증을 위한 훈련 및 테스트 인덱스를 생성합니다.
  이는 각 훈련-테스트 분할에 대해 `(train_index, test_index)` 쌍을 생성하는 제너레이터(generator)를 반환합니다.
- `train_index`: 훈련 데이터의 인덱스를 담고 있는 배열.
- `test_index`: 테스트 데이터의 인덱스를 담고 있는 배열.

### 3. 주요 속성 (Attributes)
- n_splits_: `int`
  분할의 수를 나타내며, `n_splits`와 동일합니다.
- test_size_: `float`
  사용된 테스트 세트의 크기를 나타냅니다.
- train_size_: `float`
  사용된 훈련 세트의 크기를 나타냅니다.

### 4. 주요 메소드 (Functions)
- split(X, y=None, groups=None)
  입력 데이터 `X`와 (선택적으로) 레이블 `y`에 대해 교차 검증용 훈련 및 테스트 인덱스를 생성합니다.
  - X: 입력 데이터 (특징 행렬).
  - y: 타겟 레이블 (optional).
  - groups: 샘플 그룹화 정보 (optional).

  이 메소드는 `(train_index, test_index)`를 반환하는 제너레이터입니다.

### 사용 예시
아래 코드에서 `ShuffleSplit` 객체는 `n_splits=3`으로 설정되어, 3번의 무작위 분할을 수행합니다.
각 분할에서 데이터셋의 80%가 훈련 세트로, 20%가 테스트 세트로 나누어집니다.
`random_state=42`는 재현 가능한 결과를 얻기 위해 사용됩니다.
이 방법을 사용하면 데이터셋의 섞임을 통해 훈련과 테스트 데이터가 매번 다르게 나누어지므로
모델의 일반화 성능을 보다 잘 평가할 수 있습니다.
'''

from sklearn.model_selection import ShuffleSplit
import numpy as np

# 데이터 생성
X = np.arange(10).reshape((5, 2)) # 예: 5개의 샘플, 2개의 특징
y = np.array([0, 1, 0, 1, 0])     # 예: 5개의 레이블

```

```
# ShuffleSplit 객체 생성
rs = ShuffleSplit(n_splits=3, test_size=0.2, random_state=42)

# 분할된 인덱스 확인
for train_index, test_index in rs.split(X):
    print("TRAIN:", train_index, "TEST:", test_index)
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
```

```
TRAIN: [4 2 0 3] TEST: [1]
TRAIN: [1 2 0 4] TEST: [3]
TRAIN: [0 3 4 2] TEST: [1]
```

In [11]:

```
'''
코드는 모델의 성능을 교차검증하고, 최종적으로 선택된 모델을 사용하여
테스트 데이터에 대한 예측값을 생성하는 과정을 수행합니다.

# 1. 필요한 라이브러리와 데이터 준비
우선 필요한 라이브러리와 데이터를 불러옵니다.

# 2. 데이터 로드 및 설정
데이터와 필요한 변수들을 설정합니다.
- `s_hist = []`: 각 모델의 성능 기록을 저장할 빈 리스트입니다.
- `cv = KFold(n_splits=5, random_state=123)`:
    5-fold 교차검증을 위한 `KFold` 객체를 생성합니다. 데이터를 5개의 폴드로 나누어
    각각을 검증에 사용하며, `random_state=123`으로 시드 값을 고정하여 재현성을 보장합니다.
- `ss = ShuffleSplit(n_splits=1, train_size=0.8, random_state=123)`:
    단 한 번의 훈련/검증 셋 분할을 위한 `ShuffleSplit` 객체를 생성합니다.
    전체 데이터의 80%를 훈련 데이터로, 나머지 20%를 검증 데이터로 사용합니다.
    역시 `random_state=123`으로 시드 값을 고정합니다.

- `df_ans = pd.read_csv('test_prob_ans.csv', index_col='id')`:
    테스트 데이터의 정답값을 저장한 CSV 파일을 읽어와 `df_ans` 데이터프레임에 저장합니다.
    `id` 열을 인덱스로 설정합니다.

- `X_all = df_test.columns.tolist() + ['targetA_prob']`:
    테스트 데이터의 모든 열 이름과 'targetA_prob' 열 이름을 결합하여
    `X_all` 리스트를 생성합니다. 이는 모델 학습과 예측에 사용될 모든 특성(피처)입니다.

- `cat_cols`: 'cat0'부터 'cat9'까지의 범주형 변수 이름을 저장하는 리스트입니다.
- `cont_cols`: 'cont0'부터 'cont13'까지의 연속형 변수 이름을 저장하는 리스트입니다.
- `cont_q_cols`: 'cont0_q'부터 'cont13_q'까지의 파생 변수 이름을 저장하는 리스트입니다.
```

```
# 3. 파생 변수 생성 및 모델 평가 함수 정의
이 함수는 모델을 교차검증하여 성능을 평가합니다.
- `q`: 0부터 1까지 0.01 간격의 값들을 저장하는 리스트입니다.
  이 값들은 분위수(quantile)를 계산하는 데 사용됩니다.

- `def eval_model(model_name, model, sp):`:
  모델을 교차검증하여 성능을 평가하는 함수입니다. `model_name`은 모델의 이름,
  `model`은 모델 객체, `sp`는 `KFold` 또는 `ShuffleSplit` 객체입니다.

- `score_train, score_valid = list(), list()`:
  훈련셋과 검증셋의 성능을 저장할 빈 리스트입니다.

- `for train_idx, valid_idx in sp.split(df_train):`:
  `KFold` 또는 `ShuffleSplit` 객체를 사용하여 훈련셋과 검증셋의 인덱스를 생성합니다.

- `df_cv_train, df_valid = df_train.iloc[train_idx].copy(), df_train.iloc[valid_idx].copy()`:
  훈련셋과 검증셋을 인덱스를 이용해 나눕니다.
  `.copy()`를 사용하여 원본 데이터프레임의 복사본을 생성합니다.

- 연속형 변수 파생 변수 생성:
  - `for i in range(0, 14):`: 'cont0'부터 'cont13'까지의 연속형 변수를 반복합니다.
  - `col = 'cont{}'.format(i)`: 현재 반복 변수의 이름을 생성합니다.
  - `qt = df_cv_train[col].quantile(q)`: 훈련 데이터의 해당 열에 대해 분위수를 계산합니다.
  - `qt.iloc[[0, -1]] = [-np.inf, np.inf]`:
    첫 번째와 마지막 분위수 값을 -무한대와 무한대로 설정합니다.
  - `q_range = pd.cut(df_cv_train[col], bins=qt)`:
    분위수에 따라 데이터를 구간으로 나눕니다.
  - `q_mean = df_cv_train.groupby(q_range)['target'].mean()`:
    각 구간의 타겟값의 평균을 계산합니다.
  - `df_cv_train[col + '_q'] = q_range.map(q_mean).astype('float')`:
    각 구간의 평균값을 새로운 파생 변수로 추가합니다.
  - `df_valid[col + '_q'] = pd.cut(df_valid[col], bins=qt).map(q_mean).astype('float')`:
    검증 데이터에 동일한 구간 나누기를 적용하고 평균값을 파생 변수로 추가합니다.

- 모델 학습 및 성능 평가:
  - `model.fit(df_cv_train[X_all], df_cv_train['target'])`:
    훈련 데이터를 사용하여 모델을 학습시킵니다.
  - `score_valid.append((mean_squared_error(df_valid['target'],
    model.predict(df_valid[X_all]))) 0.5)`:
    검증 데이터에 대한 예측값과 실제값의 RMSE를 계산하여 리스트에 추가합니다.
  - `score_train.append((mean_squared_error(df_cv_train['target'],
    model.predict(df_cv_train[X_all]))) 0.5)`:
    훈련 데이터에 대한 예측값과 실제값의 RMSE를 계산하여 리스트에 추가합니다.
```

```
- 결과 출력 및 저장:
- `output = 'Valid: {:.5f}±{:.5f}, V.Train: {:.5f}±{:.5f}'.format(np.mean(score_valid),
    np.std(score_valid), np.mean(score_train), np.std(score_train))`:
    검증 및 훈련 데이터의 평균 및 표준편차를 포맷팅하여 출력 문자열을 생성합니다.
- `print(output)` : 출력 문자열을 콘솔에 출력합니다.
- `s_hist.append(pd.Series([model_name, output, np.mean(score_valid)],
    index=['model name', 'result', 'score']))`:
    모델 이름, 출력 문자열, 검증 데이터의 평균 RMSE를 시리즈로 만들어 리스트에 추가합니다.
- `df_result = pd.DataFrame(s_hist)` : 리스트를 데이터프레임으로 변환합니다.
- `display(df_result.groupby('model name').last())` : 각 모델의 마지막 결과를 출력합니다.
- `display(df_result.loc[df_result['score'].idxmin()])` :
    가장 낮은 검증 점수를 가진 모델의 결과를 출력합니다.
```

4. 최종 모델 선택 함수 정의

이 함수는 최종 모델을 학습시키고, 테스트 데이터에 대한 예측값을 생성하여 파일로 저장합니다.

```
- `def select_model(model)` : 최종 모델을 학습시키고 예측값을 생성하는 함수입니다.
- `model.fit(df_train[X_all], df_train['target'])` :
    전체 훈련 데이터를 사용하여 모델을 학습시킵니다.
- `prd = model.predict(df_test[X_all])` : 테스트 데이터에 대한 예측값을 생성합니다.
- `pd.DataFrame({'id': df_test.index.values, 'target': prd}).to_csv('answer6.csv',
    index = None)` :
    예측 결과를 데이터프레임으로 만들어 'answer6.csv' 파일로 저장합니다.
- `return prd` : 예측값을 반환합니다.
```

코드 동작 설명

1. `eval\_model` 함수:

- 모델의 성능을 교차검증합니다.
- `sp`는 `ShuffleSplit` 또는 `KFold` 객체입니다.
- 각 fold마다 훈련셋과 검증셋을 나눠 파생 변수를 생성하고, 모델을 학습합니다.
- 검증셋에 대한 예측값과 실제값을 비교하여 RMSE를 계산합니다.
- 결과를 출력하고 기록합니다.

2. `select\_model` 함수:

- 최종 모델을 전체 훈련 데이터로 학습시킵니다.
- 테스트 데이터에 대한 예측값을 생성하여 `answer6.csv` 파일로 저장합니다.

주요 포인트

- 파생 변수 생성:
 - `cont0`부터 `cont13`까지의 연속형 변수에 대해 분위수 기반의 구간을 나누고,
 - 각 구간에 속하는 `target`의 평균을 계산하여 파생 변수를 생성합니다.
- 모델 학습 및 검증: 생성된 파생 변수를 포함하여 모델을 학습시키고,
 - 교차검증을 통해 검증셋에 대한 성능을 평가합니다.

- 결과 기록: 모델 성능 결과를 출력하고 기록하여, 최종적으로 가장 성능이 좋은 모델을 선택할 수 있도록 합니다.

...

공통

```
from sklearn.pipeline import make_pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.metrics import accuracy_score, mean_squared_error
from sklearn.metrics import make_scorer, mean_squared_error
from sklearn.model_selection import ShuffleSplit, KFold
```

```
cv = KFold(n_splits=5, random_state=123)
```

train(80%)/test(20%) 한 번으로 검증합니다. XGB, RF 등 오래 걸리는 모델을 위해 사용합니다.

```
ss = ShuffleSplit(n_splits=1, train_size=0.8, random_state=123)
```

```
df_ans = pd.read_csv('test_prob_ans.csv', index_col='id')
```

```
X_all = df_test.columns.tolist() + ['targetA_prob']
```

```
print('X_all = ', X_all)
```

```
print('-----')
```

```
cat_cols = ['cat{}'.format(i) for i in range(10)]
```

```
cont_cols = ['cont{}'.format(i) for i in range(14)]
```

```
cont_q_cols = ['cont{}_q'.format(i) for i in range(14)]
```

위에서 발생한 Leak을 바로 잡아 교차검증을 합니다.

```
q = [i for i in np.arange(0, 1.01, 0.01)]
```

```
def eval_model(model_name, model, sp):
```

```
    score_train, score_valid = list(), list()
```

```
    for train_idx, valid_idx in sp.split(df_train):
```

```
        df_cv_train, df_valid = df_train.iloc[train_idx].copy(), \
                                df_train.iloc[valid_idx].copy()
```

시험에서 구현하기 어려울 수 있으니, 시험 때는 leakage가 있어

성능에 측정에 왜곡이 있음을 고려하고 cross_validate를

사용해서 구성을 해도 됩니다.

검증셋에서 train으로 파생변수를 만들고

검증셋의 test(검외셋)에 검증셋의 train으로 만든 통계값(mean)을 반영합니다.

```
    for i in range(0, 14):
```

```
        col = 'cont{}'.format(i)
```

```
        qt = df_cv_train[col].quantile(q)
```

```
        qt.iloc[[0, -1]] = [-np.inf, np.inf]
```

```
        q_range = pd.cut(df_cv_train[col], bins=qt)
```

```

    q_mean = df_cv_train.groupby(q_range)['target'].mean()
    df_cv_train[col + '_q'] = q_range.map(q_mean).astype('float')
    df_valid[col + '_q'] = pd.cut(df_valid[col], bins=qt).map(q_mean).astype('float')
    model.fit(df_cv_train[X_all], df_cv_train['target'])
    score_valid.append((mean_squared_error(df_valid['target'],
                                           model.predict(df_valid[X_all]))) ** 0.5)

    score_train.append((mean_squared_error(df_cv_train['target'],
                                           model.predict(df_cv_train[X_all]))) ** 0.5)

output = 'Valid: {:.5f}±{:.5f}, V.Train: {:.5f}±{:.5f}'.format(
    np.mean(score_valid), np.std(score_valid), np.mean(score_train), np.std(score_train),
)
print(output)
s_hist.append(pd.Series([model_name, output, np.mean(score_valid)],
                        index=['model name', 'result', 'score']))
df_result = pd.DataFrame(s_hist)
display(df_result.groupby('model name').last())
display(df_result.loc[df_result['score'].idxmin()])

def select_model(model):
    model.fit(df_train[X_all], df_train['target'])
    prd = model.predict(df_test[X_all])
    pd.DataFrame({
        'id': df_test.index.values,
        'target': prd
    }).to_csv('answer6.csv', index = None)
    return prd

X_all = ['cat0', 'cat1', 'cat2', 'cat3', 'cat4', 'cat5', 'cat6', 'cat7', 'cat8', 'cat9', 'cont0', 'cont1', 'cont2', 'cont3', 'c
ont4', 'cont5', 'cont6', 'cont7', 'cont8', 'cont9', 'cont10', 'cont11', 'cont12', 'cont13', 'targetA_prob', 'cont0_q', 'cont1_
q', 'cont2_q', 'cont3_q', 'cont4_q', 'cont5_q', 'cont6_q', 'cont7_q', 'cont8_q', 'cont9_q', 'cont10_q', 'cont11_q', 'cont12_q',
'cont13_q', 'targetA_prob']
-----

```

In [12]:

```

'''
# 코드 설명
- `ct`: `ColumnTransformer` 객체를 생성합니다.
    이는 데이터 전처리를 열별로 다르게 적용할 수 있게 합니다.
- `('ohe', OneHotEncoder(drop='first'), cat_cols)`: 첫 번째 변환기입니다.
    - `ohe`: 변환기의 이름으로, 원-핫 인코딩을 수행함을 나타냅니다.
    - `OneHotEncoder(drop='first')`: 범주형 변수를 원-핫 인코딩합니다.
        `drop='first'` 옵션은 첫 번째 범주를 드롭하여 다중공선성 문제를 줄입니다.
'''

```

```

- `cat_cols`: 원-핫 인코딩을 적용할 열의 목록입니다.
- `('std', StandardScaler(), cont_cols)`: 두 번째 변환기입니다.
- `std`: 변환기의 이름으로, 표준화 작업을 수행함을 나타냅니다.
- `StandardScaler()`: 연속형 변수를 평균이 0이고 표준편차가 1이 되도록 표준화합니다.
- `cont_cols`: 표준화를 적용할 열의 목록입니다.

- `reg_lr`: `make_pipeline` 함수를 사용하여 파이프라인 객체를 생성합니다.
- `ct`: 위에서 정의한 `ColumnTransformer` 객체입니다. 데이터 전처리를 수행합니다.
- `LinearRegression()`: 선형 회귀 모델을 파이프라인의 마지막 단계로 추가합니다.
  전처리된 데이터를 사용하여 선형 회귀 모델을 학습합니다.

- `eval_model('baseline', reg_lr, cv)`:
  `eval_model` 함수를 호출하여 모델의 성능을 평가합니다.
- `'baseline'`: 평가할 모델의 이름입니다.
- `reg_lr`: 평가할 모델 객체입니다. 여기서는 `ColumnTransformer`와
  `LinearRegression`으로 구성된 파이프라인입니다.
- `cv`: 교차검증을 위한 `KFold` 객체입니다.
  5개의 폴드로 데이터를 나누어 교차검증을 수행합니다.

# `eval_model` 함수의 동작
위에서 정의한 `eval_model` 함수는 다음과 같이 동작합니다.

1. 초기화:
  - `score_train`과 `score_valid` 리스트를 초기화합니다.
    이 리스트는 각 폴드에 대한 훈련 및 검증 점수를 저장합니다.

2. 교차검증 루프:
  - `for train_idx, valid_idx in sp.split(df_train):`
    루프를 사용하여 데이터를 훈련셋과 검증셋으로 나눕니다.
  - 훈련셋에서 각 연속형 변수에 대해 파생 변수를 생성하고
    검증셋에도 동일한 변환을 적용합니다.
  - 모델을 훈련 데이터에 맞추어 학습시킵니다.
  - 훈련 데이터와 검증 데이터에 대해 예측을 수행하고,
    각 데이터셋의 RMSE를 계산하여 리스트에 추가합니다.

3. 결과 출력 및 저장:
  - 검증 및 훈련 점수의 평균과 표준편차를 계산하고 포매팅하여 출력 문자열을 생성합니다.
  - 출력 문자열을 콘솔에 출력합니다.
  - 모델의 이름과 성능 결과를 시리즈로 만들어 `s_hist` 리스트에 추가합니다.
  - 결과를 데이터프레임으로 변환하고 각 모델의 마지막 결과 및
    가장 낮은 검증 점수를 가진 모델의 결과를 출력합니다.

```

이와 같은 과정을 통해 주어진 `ColumnTransformer`와 `LinearRegression`으로 구성된 파이프라인 모델의 성능을 교차검증하고 평가할 수 있습니다.

```
'''
from sklearn.linear_model import LinearRegression

# 아무처리 없이 사용합니다.
ct = ColumnTransformer([
    ('ohe', OneHotEncoder(drop='first'), cat_cols),
    ('std', StandardScaler(), cont_cols)
])

reg_lr = make_pipeline(ct, LinearRegression())
eval_model('baseline', reg_lr, cv)
```

Valid: 0.86325±0.00295, V.Train: 0.86300±0.00073

	result	score
model name		
baseline	Valid: 0.86325±0.00295, V.Train: 0.86300±0.00073	0.863246
model name		baseline
result	Valid: 0.86325±0.00295, V.Train: 0.86300±0.00073	
score		0.863246
Name: 0, dtype: object		

In [13]:

```
'''
# 코드 설명
- `prd`: `select_model` 함수의 반환 값을 저장하는 변수입니다.
  이 변수는 테스트 데이터에 대한 예측 값을 담고 있습니다.
- `select_model(reg_lr)`: `select_model` 함수를 호출하여
  `reg_lr` 모델을 사용해 최종 예측을 수행합니다.
- `select_model` 함수는 다음과 같은 작업을 수행합니다:
  1. 모델 학습:
    - `reg_lr.fit(df_train[X_all], df_train['target'])`:
      전체 훈련 데이터를 사용하여 `reg_lr` 모델을 학습시킵니다.
  2. 예측 수행:
    - `prd = model.predict(df_test[X_all])`:
      테스트 데이터에 대한 예측 값을 생성합니다.
  3. 결과 저장:
    - `pd.DataFrame({'id': df_test.index.values, 'target': prd}).to_csv('answer6.csv',
      index = None)`:
```

```

예측 결과를 데이터프레임으로 만들어 'answer6.csv' 파일로 저장합니다.
4. 예측 값 반환:
- `return prd`: 예측 값을 반환합니다.

- `print("baseline 채점 결과:", mean_squared_error(df_ans['target'], prd) * 0.5)`:
  예측 결과를 평가하고 출력합니다.
- `mean_squared_error(df_ans['target'], prd) * 0.5`: 예측 결과의
  RMSE(Root Mean Squared Error)를 계산합니다.
- `mean_squared_error(df_ans['target'], prd)`: `df_ans['target']`과
  `prd` 간의 MSE(Mean Squared Error)를 계산합니다.
- `0.5`: MSE의 제곱근을 계산하여 RMSE를 구합니다.

# 요약
1. 모델 선택 및 예측 (`select_model` 함수 호출):
- `select_model` 함수는 주어진 모델을 전체 훈련 데이터로 학습시키고,
  테스트 데이터에 대한 예측 값을 생성하여 저장하고 반환합니다.
2. 결과 평가 및 출력:
- 반환된 예측 값을 실제 값과 비교하여 RMSE를 계산하고, 이를 출력합니다.
...

prd = select_model(reg_lr)
print("baseline 채점 결과:", mean_squared_error(df_ans['target'], prd) ** 0.5)

```

baseline 채점 결과: 0.8657267201878256

In [14]:

```

'''
### 코드 설명
- `ct`: `ColumnTransformer` 객체를 생성합니다. 이는 데이터 전처리를
  열별로 다르게 적용할 수 있게 합니다.
- `('ohe', OneHotEncoder(drop='first'), cat_cols)`: 첫 번째 변환기입니다.
  - `ohe`: 변환기의 이름으로, 원-핫 인코딩을 수행함을 나타냅니다.
  - `OneHotEncoder(drop='first')`: 범주형 변수를 원-핫 인코딩합니다.
    `drop='first'` 옵션은 첫 번째 범주를 드롭하여 다중공선성 문제를 줄입니다.
  - `cat_cols`: 원-핫 인코딩을 적용할 열의 목록입니다.
- `('std', StandardScaler(), cont_cols)`: 두 번째 변환기입니다.
  - `std`: 변환기의 이름으로, 표준화 작업을 수행함을 나타냅니다.
  - `StandardScaler()`: 연속형 변수를 평균이 0이고 표준편차가 1이 되도록 표준화합니다.
  - `cont_cols`: 표준화를 적용할 열의 목록입니다.
- `('pt', 'passthrough', ['targetA_prob'])`: 세 번째 변환기입니다.
  - `pt`: 변환기의 이름으로, 통과 작업(passthrough)을 수행함을 나타냅니다.
  - `passthrough`: `targetA_prob` 열을 변환하지 않고 그대로 통과시킵니다.
  - `['targetA_prob']`: 통과 작업을 적용할 열입니다.

```

```

- `reg_lr2`: `make_pipeline` 함수를 사용하여 파이프라인 객체를 생성합니다.
- `ct`: 위에서 정의한 `ColumnTransformer` 객체입니다. 데이터 전처리를 수행합니다.
- `LinearRegression()`: 선형 회귀 모델을 파이프라인의 마지막 단계로 추가합니다.
  전처리된 데이터를 사용하여 선형 회귀 모델을 학습합니다.

- `eval_model('lr2', reg_lr2, cv)`: `eval_model` 함수를 호출하여 모델의 성능을 평가합니다.
- `'lr2'`: 평가할 모델의 이름입니다.
- `reg_lr2`: 평가할 모델 객체입니다. 여기서는 `ColumnTransformer`와
  `LinearRegression`으로 구성된 파이프라인입니다.
- `cv`: 교차검증을 위한 `KFold` 객체입니다.
  5개의 폴드로 데이터를 나누어 교차검증을 수행합니다.

- `prd`: `select_model` 함수의 반환 값을 저장하는 변수입니다.
  이 변수는 테스트 데이터에 대한 예측 값을 담고 있습니다.
- `select_model(reg_lr2)`: `select_model` 함수를 호출하여 `reg_lr2` 모델을 사용해
  최종 예측을 수행합니다.
- `select_model` 함수는 다음과 같은 작업을 수행합니다:
  1. 모델 학습:
    - `reg_lr2.fit(df_train[X_all], df_train['target'])`:
      전체 훈련 데이터를 사용하여 `reg_lr2` 모델을 학습시킵니다.
  2. 예측 수행:
    - `prd = model.predict(df_test[X_all])`: 테스트 데이터에 대한 예측 값을 생성합니다.
  3. 결과 저장:
    - `pd.DataFrame({'id': df_test.index.values, 'target': prd}).to_csv('answer6.csv',
      index = None)`:
      예측 결과를 데이터프레임으로 만들어 'answer6.csv' 파일로 저장합니다.
  4. 예측 값 반환:
    - `return prd`: 예측 값을 반환합니다.

- `print("lr2 채점 결과:", mean_squared_error(df_ans['target'], prd) * 0.5)`:
  예측 결과를 평가하고 출력합니다.
- `"lr2 채점 결과:"`: 출력 메시지의 첫 부분입니다.
- `mean_squared_error(df_ans['target'], prd) * 0.5`:
  예측 결과의 RMSE(Root Mean Squared Error)를 계산합니다.
- `mean_squared_error(df_ans['target'], prd)`:
  `df_ans['target']`과 `prd` 간의 MSE(Mean Squared Error)를 계산합니다.
- `0.5`: MSE의 제곱근을 계산하여 RMSE를 구합니다.
- 최종적으로, `print` 함수는 "lr2 채점 결과: {RMSE 값}" 형태의 문자열을 출력합니다.

```

요약

1. 데이터 전처리 구성 (`ColumnTransformer`):

- 범주형 변수는 원-핫 인코딩하고, 연속형 변수는 표준화하며, ``targetA_prob`` 변수는 그대로 사용합니다.
2. 모델 생성 (``make_pipeline``):
 - 전처리 구성과 선형 회귀 모델을 파이프라인으로 연결합니다.
 3. 모델 평가 (``eval_model`` 함수 호출):
 - 교차검증을 통해 모델의 성능을 평가합니다.
 4. 모델 선택 및 예측 (``select_model`` 함수 호출):
 - 전체 훈련 데이터로 모델을 학습시키고, 테스트 데이터에 대한 예측을 수행합니다.
 5. 결과 평가 및 출력:
 - 예측 값을 실제 값과 비교하여 RMSE를 계산하고, 이를 출력합니다.

이와 같이, 주어진 코드는 추가적인 변수(``targetA_prob``)를 포함한 데이터 전처리와 선형 회귀 모델을 통해 최종 예측을 수행하고, 예측 결과를 평가하여 출력하는 과정을 포함하고 있습니다.

```
'''
# + targetA_prob 를 추가합니다.
ct = ColumnTransformer([
    ('ohe', OneHotEncoder(drop='first'), cat_cols),
    ('std', StandardScaler(), cont_cols),
    ('pt', 'passthrough', ['targetA_prob'])
])

reg_lr2 = make_pipeline(ct, LinearRegression())
eval_model('lr2', reg_lr2, cv)
```

Valid: 0.84697±0.00283, V.Train: 0.84668±0.00071

	result	score
model name		
baseline	Valid: 0.86325±0.00295, V.Train: 0.86300±0.00073	0.863246
lr2	Valid: 0.84697±0.00283, V.Train: 0.84668±0.00071	0.846973

```
model name                                lr2
result      Valid: 0.84697±0.00283, V.Train: 0.84668±0.00071
score                                             0.846973
Name: 1, dtype: object
```

```
In [15]: prd = select_model(reg_lr2)
print("lr2 채점 결과:", mean_squared_error(df_ans['target'], prd) ** 0.5)
```

lr2 채점 결과: 0.8491929478687422

In [16]:

```
'''
물론입니다. 주어진 코드를 라인별로 자세히 설명해드리겠습니다.

### 코드 설명

- `ct`: `ColumnTransformer` 객체를 생성합니다.
    이는 데이터 전처리를 열별로 다르게 적용할 수 있게 합니다.
- `('ohe', OneHotEncoder(drop='first'), cat_cols)`: 첫 번째 변환기입니다.
    - `ohe`: 변환기의 이름으로, 원-핫 인코딩을 수행함을 나타냅니다.
    - `OneHotEncoder(drop='first')`: 범주형 변수를 원-핫 인코딩합니다.
        `drop='first'` 옵션은 첫 번째 범주를 드롭하여 다중공선성 문제를 줄입니다.
    - `cat_cols`: 원-핫 인코딩을 적용할 열의 목록입니다.
- `('std', StandardScaler(), cont_q_cols)`: 두 번째 변환기입니다.
    - `std`: 변환기의 이름으로, 표준화 작업을 수행함을 나타냅니다.
    - `StandardScaler()`: 연속형 변수를 평균이 0이고 표준편차가 1이 되도록 표준화합니다.
    - `cont_q_cols`: 표준화를 적용할 파생 변수(`cont_q` 열)의 목록입니다.
- `('pt', 'passthrough', ['targetA_prob'])`: 세 번째 변환기입니다.
    - `pt`: 변환기의 이름으로, 통과 작업(passthrough)을 수행함을 나타냅니다.
    - `passthrough`: `targetA_prob` 열을 변환하지 않고 그대로 통과시킵니다.
    - `[targetA_prob]`: 통과 작업을 적용할 열입니다.

- `reg_lr3`: `make_pipeline` 함수를 사용하여 파이프라인 객체를 생성합니다.
    - `ct`: 위에서 정의한 `ColumnTransformer` 객체입니다. 데이터 전처리를 수행합니다.
    - `LinearRegression()`: 선형 회귀 모델을 파이프라인의 마지막 단계로 추가합니다.
        전처리된 데이터를 사용하여 선형 회귀 모델을 학습합니다.

- `eval_model('lr3', reg_lr3, cv)`: `eval_model` 함수를 호출하여 모델의 성능을 평가합니다.
    - `lr3`: 평가할 모델의 이름입니다.
    - `reg_lr3`: 평가할 모델 객체입니다. 여기서는 `ColumnTransformer`와
        `LinearRegression`으로 구성된 파이프라인입니다.
    - `cv`: 교차검증을 위한 `KFold` 객체입니다.
        5개의 폴드로 데이터를 나누어 교차검증을 수행합니다.

### `eval_model` 함수 내부 동작 설명
- 모델 학습 및 검증:
    - `eval_model` 함수는 주어진 데이터에서 교차 검증을 통해 모델의 성능을 평가합니다.
    - `cv.split(df_train)`을 통해 훈련 데이터셋을 KFold 교차검증의 폴드로 나눕니다.
    - 각 폴드에서 훈련 데이터와 검증 데이터를 나눠 전처리를 적용하고 모델을 학습합니다.
    - 검증 데이터에 대해 예측을 수행하고, 예측값과 실제값의 RMSE를 계산합니다.
- 평가 결과 출력:
```



```

- 각 폴드의 검증 결과와 훈련 결과를 저장하고, 평균 및 표준편차를 출력합니다.
- 결과를 `s_hist` 리스트에 저장하여 최종 모델 성능을 비교할 수 있게 합니다.

- `prd`: `select_model` 함수의 반환 값을 저장하는 변수입니다.
  이 변수는 테스트 데이터에 대한 예측 값을 담고 있습니다.
- `select_model(reg_lr3)`:
  `select_model` 함수를 호출하여 `reg_lr3` 모델을 사용해 최종 예측을 수행합니다.
- `select_model` 함수는 다음과 같은 작업을 수행합니다:
  1. 모델 학습:
    - `reg_lr3.fit(df_train[X_all], df_train['target'])`:
      전체 훈련 데이터를 사용하여 `reg_lr3` 모델을 학습시킵니다.
  2. 예측 수행:
    - `prd = model.predict(df_test[X_all])`: 테스트 데이터에 대한 예측 값을 생성합니다.
  3. 결과 저장:
    - `pd.DataFrame({'id': df_test.index.values, 'target': prd}).to_csv('answer6.csv',
      index = None)`:
      예측 결과를 데이터프레임으로 만들어 'answer6.csv' 파일로 저장합니다.
  4. 예측 값 반환:
    - `return prd`: 예측 값을 반환합니다.

- `print("lr3 채점 결과:", mean_squared_error(df_ans['target'], prd) 0.5)`:
  예측 결과를 평가하고 출력합니다.
- `"lr3 채점 결과:"`: 출력 메시지의 첫 부분입니다.
- `mean_squared_error(df_ans['target'], prd) 0.5`:
  예측 결과의 RMSE(Root Mean Squared Error)를 계산합니다.
- `mean_squared_error(df_ans['target'], prd)`: `df_ans['target']`과
  `prd` 간의 MSE(Mean Squared Error)를 계산합니다.
- `0.5`: MSE의 제곱근을 계산하여 RMSE를 구합니다.
- 최종적으로, `print` 함수는 "lr3 채점 결과: {RMSE 값}" 형태의 문자열을 출력합니다.

### 요약
1. 데이터 전처리 구성 (`ColumnTransformer`):
  - 범주형 변수는 원-핫 인코딩하고, 파생된 연속형 변수(`cont_q` 열)는 표준화하며,
    `targetA_prob` 변수는 그대로 사용합니다.
2. 모델 생성 (`make_pipeline`):
  - 전처리 구성과 선형 회귀 모델을 파이프라인으로 연결합니다.
3. 모델 평가 (`eval_model` 함수 호출):
  - 교차검증을 통해 모델의 성능을 평가합니다.
4. 모델 선택 및 예측 (`select_model` 함수 호출):
  - 전체 훈련 데이터로 모델을 학습시키고, 테스트 데이터에 대한 예측을 수행합니다.
5. 결과 평가 및 출력:
  - 예측 값을 실제 값과 비교하여 RMSE를 계산하고, 이를 출력합니다.

```

이와 같이, 주어진 코드는 추가적인 변수(`targetA\_prob`)와 파생 변수(`cont\_q` 열)를 포함한 데이터 전처리와 선형 회귀 모델을 통해 최종 예측을 수행하고, 예측 결과를 평가하여 출력하는 과정을 포함하고 있습니다.

```
'''
# + targetA_prob 를 추가합니다.
# + cont{}_q 를 cont{} 대신에 사용합니다.

ct = ColumnTransformer([
    ('ohe', OneHotEncoder(drop='first'), cat_cols),
    ('std', StandardScaler(), cont_q_cols),
    ('pt', 'passthrough', ['targetA_prob'])
])

reg_lr3 = make_pipeline(ct, LinearRegression())
eval_model('lr3', reg_lr3, cv)
```

Valid: 0.84651±0.00256, V.Train: 0.84370±0.00137

	result	score
model name		
baseline	Valid: 0.86325±0.00295, V.Train: 0.86300±0.00073	0.863246
lr2	Valid: 0.84697±0.00283, V.Train: 0.84668±0.00071	0.846973
lr3	Valid: 0.84651±0.00256, V.Train: 0.84370±0.00137	0.846513

```
model name          lr3
result      Valid: 0.84651±0.00256, V.Train: 0.84370±0.00137
score                                0.846513
Name: 2, dtype: object
```

```
In [17]: prd = select_model(reg_lr3)
print("lr3 채점 결과:", mean_squared_error(df_ans['target'], prd) ** 0.5)
```

lr3 채점 결과: 0.8482386126039785

```
In [18]: '''
### 코드 설명

- `ct`: `ColumnTransformer` 객체를 생성합니다.
  이는 데이터 전처리를 열별로 다르게 적용할 수 있게 합니다.
```

```

- `('ohe', OneHotEncoder(drop='first'), cat_cols)`: 첫 번째 변환기입니다.
- `ohe`: 변환기의 이름으로, 원-핫 인코딩을 수행함을 나타냅니다.
- `OneHotEncoder(drop='first')`: 범주형 변수를 원-핫 인코딩합니다.
  `drop='first'` 옵션은 첫 번째 범주를 드롭하여 다중공선성 문제를 줄입니다.
- `cat_cols`: 원-핫 인코딩을 적용할 열의 목록입니다.
- `('std', make_pipeline(StandardScaler(), PCA(n_components=0.95)), cont_q_cols)`:
  두 번째 변환기입니다.
- `std`: 변환기의 이름으로, 표준화 및 PCA를 수행함을 나타냅니다.
- `make_pipeline(StandardScaler(), PCA(n_components=0.95))`:
  연속형 변수(`cont_q` 열)에 대해 표준화를 수행한 후, 주성분 분석(PCA)을 적용합니다.
- `StandardScaler()`: 연속형 변수를 평균이 0이고 표준편차가 1이 되도록 표준화합니다.
- `PCA(n_components=0.95)`:
  주성분 분석(PCA)을 통해 데이터의 95% 분산을 설명할 수 있는 주성분을 선택합니다.
- `cont_q_cols`: 표준화 및 PCA를 적용할 파생 변수(`cont_q` 열)의 목록입니다.
- `('pt', 'passthrough', ['targetA_prob'])`: 세 번째 변환기입니다.
- `pt`: 변환기의 이름으로, 통과 작업(passthrough)을 수행함을 나타냅니다.
- `passthrough`: `targetA_prob` 열을 변환하지 않고 그대로 통과시킵니다.
- `[targetA_prob]`: 통과 작업을 적용할 열입니다.

- `reg_lr4`: `make_pipeline` 함수를 사용하여 파이프라인 객체를 생성합니다.
- `ct`: 위에서 정의한 `ColumnTransformer` 객체입니다. 데이터 전처리를 수행합니다.
- `LinearRegression()`: 선형 회귀 모델을 파이프라인의 마지막 단계로 추가합니다.
  전처리된 데이터를 사용하여 선형 회귀 모델을 학습합니다.

- `eval_model('lr4', reg_lr4, cv)`: `eval_model` 함수를 호출하여 모델의 성능을 평가합니다.
- `lr4`: 평가할 모델의 이름입니다.
- `reg_lr4`: 평가할 모델 객체입니다. 여기서는 `ColumnTransformer`와
  `LinearRegression`으로 구성된 파이프라인입니다.
- `cv`: 교차검증을 위한 `KFold` 객체입니다.
  5개의 폴드로 데이터를 나누어 교차검증을 수행합니다.

### `eval_model` 함수 내부 동작 설명
- 모델 학습 및 검증:
  - `eval_model` 함수는 주어진 데이터에서 교차 검증을 통해 모델의 성능을 평가합니다.
  - `cv.split(df_train)`을 통해 훈련 데이터셋을 KFold 교차검증의 폴드로 나눕니다.
  - 각 폴드에서 훈련 데이터와 검증 데이터를 나눠 전처리를 적용하고 모델을 학습합니다.
  - 검증 데이터에 대해 예측을 수행하고, 예측값과 실제값의 RMSE를 계산합니다.
- 평가 결과 출력:
  - 각 폴드의 검증 결과와 훈련 결과를 저장하고, 평균 및 표준편차를 출력합니다.
  - 결과를 `s_hist` 리스트에 저장하여 최종 모델 성능을 비교할 수 있게 합니다.

- `prd`: `select_model` 함수의 반환 값을 저장하는 변수입니다.

```

이 변수는 테스트 데이터에 대한 예측 값을 담고 있습니다.

- ``select_model(reg_lr4)`: `select_model` 함수를 호출하여 `reg_lr4` 모델을 사용해 최종 예측을 수행합니다.`
- ``select_model`` 함수는 다음과 같은 작업을 수행합니다:
 1. 모델 학습:
 - ``reg_lr4.fit(df_train[X_all], df_train['target'])``: 전체 훈련 데이터를 사용하여 ``reg_lr4`` 모델을 학습시킵니다.
 2. 예측 수행:
 - ``prd = model.predict(df_test[X_all])``: 테스트 데이터에 대한 예측 값을 생성합니다.
 3. 결과 저장:
 - ``pd.DataFrame({'id': df_test.index.values, 'target': prd}).to_csv('answer6.csv', index = None)``: 예측 결과를 데이터프레임으로 만들어 `'answer6.csv'` 파일로 저장합니다.
 4. 예측 값 반환:
 - ``return prd``: 예측 값을 반환합니다.
- ``print("lr4 채점 결과:", mean_squared_error(df_ans['target'], prd) 0.5)``: 예측 결과를 평가하고 출력합니다.
- ``"lr4 채점 결과:"``: 출력 메시지의 첫 부분입니다.
- ``mean_squared_error(df_ans['target'], prd) 0.5``: 예측 결과의 RMSE(Root Mean Squared Error)를 계산합니다.
- ``mean_squared_error(df_ans['target'], prd)``: ``df_ans['target']``과 ``prd`` 간의 MSE(Mean Squared Error)를 계산합니다.
- ``0.5``: MSE의 제곱근을 계산하여 RMSE를 구합니다.
- 최종적으로, ``print`` 함수는 `"lr4 채점 결과: {RMSE 값}"` 형태의 문자열을 출력합니다.

요약

1. 데이터 전처리 구성 (``ColumnTransformer``):
 - 범주형 변수는 원-핫 인코딩하고, 파생된 연속형 변수(``cont_q`` 열)는 표준화 및 PCA를 적용하며, ``targetA_prob`` 변수는 그대로 사용합니다.
2. 모델 생성 (``make_pipeline``):
 - 전처리 구성과 선형 회귀 모델을 파이프라인으로 연결합니다.
3. 모델 평가 (``eval_model`` 함수 호출):
 - 교차검증을 통해 모델의 성능을 평가합니다.
4. 모델 선택 및 예측 (``select_model`` 함수 호출):
 - 전체 훈련 데이터로 모델을 학습시키고, 테스트 데이터에 대한 예측을 수행합니다.
5. 결과 평가 및 출력:
 - 예측 값을 실제 값과 비교하여 RMSE를 계산하고, 이를 출력합니다.

이와 같이, 주어진 코드는 추가적인 변수(``targetA_prob``)와 파생 변수(``cont_q`` 열)를 포함한 데이터 전처리와 선형 회귀 모델을 통해 최종 예측을 수행하고, 예측 결과를 평가하여 출력하는 과정을 포함하고 있습니다.

```
'''
# + targetA_prob 를 추가합니다.
# PCA + cont{}_q 를 cont{} 대신에 사용합니다.

from sklearn.linear_model import LinearRegression
from sklearn.decomposition import PCA

ct = ColumnTransformer([
    ('ohe', OneHotEncoder(drop='first'), cat_cols),
    ('std', make_pipeline(StandardScaler(), PCA(n_components=0.95)), cont_q_cols),
    ('pt', 'passthrough', ['targetA_prob'])
])
reg_lr4 = make_pipeline(
    ct,
    LinearRegression()
)
eval_model('lr4', reg_lr4, cv)
```

Valid: 0.84614±0.00290, V.Train: 0.84329±0.00061

	result	score
model name		
baseline	Valid: 0.86325±0.00295, V.Train: 0.86300±0.00073	0.863246
lr2	Valid: 0.84697±0.00283, V.Train: 0.84668±0.00071	0.846973
lr3	Valid: 0.84651±0.00256, V.Train: 0.84370±0.00137	0.846513
lr4	Valid: 0.84614±0.00290, V.Train: 0.84329±0.00061	0.846145

```
model name                                lr4
result      Valid: 0.84614±0.00290, V.Train: 0.84329±0.00061
score                                             0.846145
Name: 3, dtype: object
```

```
In [19]: prd = select_model(reg_lr4)
print("lr4 채점 결과:", mean_squared_error(df_ans['target'], prd) ** 0.5)
```

lr4 채점 결과: 0.8483414624960025

```
In [20]: '''
주어진 코드는 `RandomForestRegressor`를 사용하여 모델을 학습하고 평가하는 과정입니다.
```

코드 설명

- ``ct`: `ColumnTransformer` 객체를 생성합니다.`
- 이는 데이터 전처리를 열별로 다르게 적용할 수 있게 합니다.
- ``('ohe', OneHotEncoder(), cat_cols)``: 첫 번째 변환기입니다.
 - ``ohe``: 변환기의 이름으로, 원-핫 인코딩을 수행함을 나타냅니다.
 - ``OneHotEncoder()``: 범주형 변수를 원-핫 인코딩합니다.
 - ``cat_cols``: 원-핫 인코딩을 적용할 열의 목록입니다.
- ``('pt', 'passthrough', cont_cols + ['targetA_prob'])``: 두 번째 변환기입니다.
 - ``pt``: 변환기의 이름으로, 통과 작업(`passthrough`)을 수행함을 나타냅니다.
 - ``'passthrough'``: 연속형 변수(``cont_cols` 열`)와 추가 변수(``targetA_prob``)를 변환하지 않고 그대로 통과시킵니다.
 - ``cont_cols + ['targetA_prob']``: 통과 작업을 적용할 열의 목록입니다.
- ``reg_rf`: `make_pipeline` 함수를 사용하여 파이프라인 객체를 생성합니다.`
- ``ct``: 위에서 정의한 ``ColumnTransformer` 객체입니다. 데이터 전처리를 수행합니다.`
- ``RandomForestRegressor(n_estimators=50, max_depth=4, random_state=123, n_jobs=4)``:
 - 랜덤 포레스트 회귀 모델을 파이프라인의 마지막 단계로 추가합니다.
 - ``n_estimators=50``: 랜덤 포레스트에 포함될 결정 트리의 개수입니다.
 - ``max_depth=4``: 각 결정 트리의 최대 깊이입니다.
 - ``random_state=123``: 난수 시드를 고정하여 결과의 재현성을 보장합니다.
 - ``n_jobs=4``: 4개의 CPU 코어를 사용하여 병렬 처리를 수행합니다.
 - 이는 학습 속도를 높이는 데 도움이 됩니다.
- ``eval_model('rf', reg_rf, ss)``:
 - ``eval_model`` 함수를 호출하여 모델의 성능을 평가합니다.
 - ``'rf'``: 평가할 모델의 이름입니다.
 - ``reg_rf``: 평가할 모델 객체입니다. 여기서는 ``ColumnTransformer`와 `RandomForestRegressor`로 구성된 파이프라인입니다.`
 - ``ss``: 홀드아웃 검증을 위한 ``ShuffleSplit` 객체입니다.`
 - 데이터를 80% 훈련셋과 20% 검증셋으로 한 번 나눕니다.
 - ``ShuffleSplit(n_splits=1, train_size=0.8, random_state=123)``:
 - 데이터셋을 한 번만 나누어 훈련과 검증을 수행합니다.
 - ``n_splits=1``: 데이터 분할 횟수는 1번입니다.
 - ``train_size=0.8``: 훈련 데이터의 비율을 80%로 설정합니다.
 - ``random_state=123``: 난수 시드를 고정하여 결과의 재현성을 보장합니다.

`eval\_model` 함수 내부 동작 설명

- 모델 학습 및 검증:
 - ``eval_model`` 함수는 주어진 데이터에서 홀드아웃 검증을 통해 모델의 성능을 평가합니다.
 - ``ss.split(df_train)``을 통해 훈련 데이터셋을 80% 훈련 데이터와

20% 검증 데이터로 나눕니다.

- 각 검증셋에서 훈련 데이터와 검증 데이터를 나눠 전처리를 적용하고 모델을 학습합니다.
- 검증 데이터에 대해 예측을 수행하고, 예측값과 실제값의 RMSE를 계산합니다.
- 평가 결과 출력:
 - 각 검증 결과와 훈련 결과를 저장하고, 평균 및 표준편차를 출력합니다.
 - 결과를 `s\_hist` 리스트에 저장하여 최종 모델 성능을 비교할 수 있게 합니다.

요약

1. 데이터 전처리 구성 (`ColumnTransformer`):

- 범주형 변수는 원-핫 인코딩하고, 연속형 변수와 추가 변수(`targetA\_prob`)는 그대로 사용합니다.

2. 모델 생성 (`make\_pipeline`):

- 전처리 구성과 랜덤 포레스트 회귀 모델을 파이프라인으로 연결합니다.

3. 모델 평가 (`eval\_model` 함수 호출):

- 홀드아웃 검증을 통해 모델의 성능을 평가합니다.

이와 같이, 주어진 코드는 `RandomForestRegressor`를 사용하여 모델을 학습하고 평가하는 과정을 포함하고 있으며, 데이터 전처리, 모델 생성, 모델 평가의 단계를 통해 최종 성능을 확인합니다.

'''

```
from sklearn.ensemble import RandomForestRegressor
```

```
ct = ColumnTransformer([
    ('ohe', OneHotEncoder(), cat_cols),
    ('pt', 'passthrough', cont_cols + ['targetA_prob'])
])
```

```
reg_rf = make_pipeline(
    ct,
    # Kaggle에서는 멀티프로세싱을 해도 됩니다.
    RandomForestRegressor(n_estimators=50, max_depth=4, random_state=123, n_jobs=4)
)
```

RandomForestRegressor는 학습에 오래걸립니다.

검증 방법을 교차검증에서 Hold Out (Shuffle Split n_splits=1) 검증으로 바꿉니다.

```
eval_model('rf', reg_rf, ss)
```

Valid: 0.84655±0.00000, V.Train: 0.84634±0.00000

	result	score
model name		
baseline	Valid: 0.86325±0.00295, V.Train: 0.86300±0.00073	0.863246
lr2	Valid: 0.84697±0.00283, V.Train: 0.84668±0.00071	0.846973
lr3	Valid: 0.84651±0.00256, V.Train: 0.84370±0.00137	0.846513
lr4	Valid: 0.84614±0.00290, V.Train: 0.84329±0.00061	0.846145
rf	Valid: 0.84655±0.00000, V.Train: 0.84634±0.00000	0.846550
model name		lr4
result	Valid: 0.84614±0.00290, V.Train: 0.84329±0.00061	
score		0.846145
Name: 3, dtype: object		

In [21]:

```
'''
주어진 코드는 `XGBRegressor`를 사용하여 모델을 학습하고 평가하는 과정입니다.

### 코드 설명

- `ct`: `ColumnTransformer` 객체를 생성합니다.
  이는 데이터 전처리를 열별로 다르게 적용할 수 있게 합니다.
- `('ohe', OneHotEncoder(), cat_cols)`: 첫 번째 변환기입니다.
  - `ohe`: 변환기의 이름으로, 원-핫 인코딩을 수행함을 나타냅니다.
  - `OneHotEncoder()`: 범주형 변수를 원-핫 인코딩합니다.
  - `cat_cols`: 원-핫 인코딩을 적용할 열의 목록입니다.
- `('pt', 'passthrough', cont_cols + ['targetA_prob'])`: 두 번째 변환기입니다.
  - `pt`: 변환기의 이름으로, 통과 작업(passthrough)을 수행함을 나타냅니다.
  - `passthrough`: 연속형 변수(`cont_cols` 열)와 추가 변수(`targetA_prob`)를
    변환하지 않고 그대로 통과시킵니다.
  - `cont_cols + ['targetA_prob']`: 통과 작업을 적용할 열의 목록입니다.

- `reg_xgb`: `make_pipeline` 함수를 사용하여 파이프라인 객체를 생성합니다.
- `ct`: 위에서 정의한 `ColumnTransformer` 객체입니다. 데이터 전처리를 수행합니다.
- `xgb.XGBRegressor(n_estimators=150, max_depth=2, random_state=123, n_jobs=4)`:
  XGBoost 회귀 모델을 파이프라인의 마지막 단계로 추가합니다.
  - `n_estimators=150`: 부스팅 라운드의 수입니다. 즉, 150개의 트리를 만듭니다.
  - `max_depth=2`: 각 트리의 최대 깊이입니다.
  - `random_state=123`: 난수 시드를 고정하여 결과의 재현성을 보장합니다.
  - `n_jobs=4`: 4개의 CPU 코어를 사용하여 병렬 처리를 수행합니다.
```


이는 학습 속도를 높이는 데 도움이 됩니다.

```
- `eval_model('xgb', reg_xgb, ss)`: `eval_model` 함수를 호출하여
  모델의 성능을 평가합니다.
- `'xgb'`: 평가할 모델의 이름입니다.
- `reg_xgb`: 평가할 모델 객체입니다. 여기서는 `ColumnTransformer`와
  `XGBRegressor`로 구성된 파이프라인입니다.
- `ss`: 홀드아웃 검증을 위한 `ShuffleSplit` 객체입니다. 데이터를 80% 훈련셋과
  20% 검증셋으로 한 번 나눕니다.
- `ShuffleSplit(n_splits=1, train_size=0.8, random_state=123)`:
  데이터셋을 한 번만 나누어 훈련과 검증을 수행합니다.
  - `n_splits=1`: 데이터 분할 횟수는 1번입니다.
  - `train_size=0.8`: 훈련 데이터의 비율을 80%로 설정합니다.
  - `random_state=123`: 난수 시드를 고정하여 결과의 재현성을 보장합니다.
```

`eval\_model` 함수 내부 동작 설명

```
- 모델 학습 및 검증:
  - `eval_model` 함수는 주어진 데이터에서 홀드아웃 검증을 통해 모델의 성능을 평가합니다.
  - `ss.split(df_train)`을 통해 훈련 데이터셋을 80% 훈련 데이터와
    20% 검증 데이터로 나눕니다.
  - 각 검증셋에서 훈련 데이터와 검증 데이터를 나눠 전처리를 적용하고 모델을 학습합니다.
  - 검증 데이터에 대해 예측을 수행하고, 예측값과 실제값의 RMSE를 계산합니다.
- 평가 결과 출력:
  - 각 검증 결과와 훈련 결과를 저장하고, 평균 및 표준편차를 출력합니다.
  - 결과를 `s_hist` 리스트에 저장하여 최종 모델 성능을 비교할 수 있게 합니다.
```

```
- `prd = select_model(reg_xgb)`: `select_model` 함수를 호출하여 최종 모델을 선택하고
  테스트 데이터에 대한 예측값을 생성합니다.
- `reg_xgb`: 최종 모델로 선택한 `XGBRegressor` 객체입니다.
- `select_model` 함수: 주어진 모델을 사용하여 훈련 데이터 전체를 학습시키고
  테스트 데이터에 대한 예측값을 생성합니다.
- `prd`: 테스트 데이터에 대한 예측값입니다.
```

```
- `print("xgb 채점 결과:", mean_squared_error(df_ans['target'], prd) 0.5)`:
  테스트 데이터에 대한 모델의 예측 성능을 평가하고 출력합니다.
- `mean_squared_error(df_ans['target'], prd) 0.5`:
  실제값과 예측값 간의 평균 제곱 오차(RMSE)를 계산합니다.
- `df_ans['target']`: 테스트 데이터의 실제값입니다.
- `prd`: 테스트 데이터에 대한 모델의 예측값입니다.
```

요약

1. 데이터 전처리 구성 (`ColumnTransformer`):

- 범주형 변수는 원-핫 인코딩하고, 연속형 변수와 추가 변수(`targetA\_prob`)는 그대로 사용합니다.
2. 모델 생성 (`make\_pipeline`):
 - 전처리 구성과 XGBoost 회귀 모델을 파이프라인으로 연결합니다.
 3. 모델 평가 (`eval\_model` 함수 호출):
 - 홀드아웃 검증을 통해 모델의 성능을 평가합니다.
 4. 최종 모델 선택 및 예측 (`select\_model` 함수 호출):
 - 최종 모델을 선택하고 테스트 데이터에 대한 예측값을 생성합니다.
 5. 모델 성능 평가 및 출력:
 - 테스트 데이터에 대한 모델의 RMSE를 계산하여 성능을 평가하고 출력합니다.

이와 같이, 주어진 코드는 `XGBRegressor`를 사용하여 모델을 학습하고 평가하는 과정을 포함하고 있으며, 데이터 전처리, 모델 생성, 모델 평가, 최종 모델 선택 및 성능 평가의 단계를 통해 최종 성능을 확인합니다.

```
'''
```

```
ct = ColumnTransformer([
    ('ohe', OneHotEncoder(), cat_cols),
    ('pt', 'passthrough', cont_cols + ['targetA_prob'])
])
reg_xgb = make_pipeline(
    ct,
    xgb.XGBRegressor(n_estimators=150, max_depth=2, random_state=123, n_jobs=4)
)
# Hold Out (Shuffle Split n_splits=1) 검증을 사용합니다.
eval_model('xgb', reg_xgb, ss)
```

Valid: 0.84615±0.00000, V.Train: 0.84477±0.00000

	result	score
model name		
baseline	Valid: 0.86325±0.00295, V.Train: 0.86300±0.00073	0.863246
lr2	Valid: 0.84697±0.00283, V.Train: 0.84668±0.00071	0.846973
lr3	Valid: 0.84651±0.00256, V.Train: 0.84370±0.00137	0.846513
lr4	Valid: 0.84614±0.00290, V.Train: 0.84329±0.00061	0.846145
rf	Valid: 0.84655±0.00000, V.Train: 0.84634±0.00000	0.846550
xgb	Valid: 0.84615±0.00000, V.Train: 0.84477±0.00000	0.846149

```

model name      lr4
result          Valid: 0.84614±0.00290, V.Train: 0.84329±0.00061
score           0.846145
Name: 3, dtype: object

```

```

In [22]: # 한 번 채점은 해봅니다.
prd = select_model(reg_xgb)
print("xgb 채점 결과:", mean_squared_error(df_ans['target'], prd) ** 0.5)

```

xgb 채점 결과: 0.8480732829872999

```

In [23]: '''
주어진 코드는 여러 개의 개별 모델을 결합하여 예측을 수행하는
`VotingRegressor`를 사용합니다.
이 앙상블 모델은 각 모델의 예측을 평균하여 최종 예측을 생성합니다.

### 코드 설명

- `reg_vt`: `VotingRegressor` 객체를 생성합니다.
- `VotingRegressor`: 여러 회귀 모델의 예측을 결합하여 최종 예측을 생성하는
  앙상블 회귀 모델입니다.
- `[('lr3', reg_lr3), ('lr4', reg_lr4), ('rf', reg_rf), ('xgb', reg_xgb)]:`
  사용할 모델들의 리스트입니다.
  - `('lr3', reg_lr3)`: `lr3` 모델 (3번째 선형 회귀 모델)입니다.
  - `('lr4', reg_lr4)`: `lr4` 모델 (4번째 선형 회귀 모델)입니다.
  - `('rf', reg_rf)`: 랜덤 포레스트 회귀 모델입니다.
  - `('xgb', reg_xgb)`: XGBoost 회귀 모델입니다.
- 주석 처리된 `reg_lr` 및 `reg_lr2`는 앙상블에 포함되지 않습니다.

- `eval_model('vt', reg_vt, ss)`:
  `eval_model` 함수를 호출하여 `VotingRegressor` 모델의 성능을 평가합니다.
- `vt`: 평가할 모델의 이름입니다.
- `reg_vt`: 평가할 모델 객체입니다. 여기서는 여러 개의 모델을 결합한
  `VotingRegressor`입니다.
- `ss`: 홀드아웃 검증을 위한 `ShuffleSplit` 객체입니다.
  데이터를 80% 훈련셋과 20% 검증셋으로 한 번 나눕니다.

### `eval_model` 함수 내부 동작 설명
- 모델 학습 및 검증:
  - `eval_model` 함수는 주어진 데이터에서 홀드아웃 검증을 통해
    모델의 성능을 평가합니다.
  - `ss.split(df_train)`을 통해 훈련 데이터셋을 80% 훈련 데이터와

```

- 20% 검증 데이터로 나눕니다.
- 각 검증셋에서 훈련 데이터와 검증 데이터를 나눠 전처리를 적용하고 모델을 학습합니다.
 - 검증 데이터에 대해 예측을 수행하고, 예측값과 실제값의 RMSE를 계산합니다.
 - 평가 결과 출력:
 - 각 검증 결과와 훈련 결과를 저장하고, 평균 및 표준편차를 출력합니다.
 - 결과를 `s\_hist` 리스트에 저장하여 최종 모델 성능을 비교할 수 있게 합니다.
 - `prd = select\_model(reg\_vt)` : `select\_model` 함수를 호출하여 최종 모델을 선택하고 테스트 데이터에 대한 예측값을 생성합니다.
 - `reg\_vt` : 최종 모델로 선택한 `VotingRegressor` 객체입니다.
 - `select\_model` 함수 : 주어진 모델을 사용하여 훈련 데이터 전체를 학습시키고 테스트 데이터에 대한 예측값을 생성합니다.
 - `prd` : 테스트 데이터에 대한 예측값입니다.
 - `print("Voting 채점 결과:", mean\_squared\_error(df\_ans['target'], prd) \* 0.5)` : 테스트 데이터에 대한 모델의 예측 성능을 평가하고 출력합니다.
 - `mean\_squared\_error(df\_ans['target'], prd) \* 0.5` : 실제값과 예측값 간의 평균 제곱 오차(RMSE)를 계산합니다.
 - `df\_ans['target']` : 테스트 데이터의 실제값입니다.
 - `prd` : 테스트 데이터에 대한 모델의 예측값입니다.

요약

1. 앙상블 모델 생성 (`VotingRegressor`):
 - `VotingRegressor` 객체를 생성하고, 여러 회귀 모델을 결합합니다.
2. 모델 평가 (`eval\_model` 함수 호출):
 - 홀드아웃 검증을 통해 `VotingRegressor` 모델의 성능을 평가합니다.
3. 최종 모델 선택 및 예측 (`select\_model` 함수 호출):
 - 최종 모델을 선택하고 테스트 데이터에 대한 예측값을 생성합니다.
4. 모델 성능 평가 및 출력:
 - 테스트 데이터에 대한 모델의 RMSE를 계산하여 성능을 평가하고 출력합니다.

이와 같이, 주어진 코드는 여러 개의 회귀 모델을 결합하여 예측 성능을 향상시키기 위해 `VotingRegressor`를 사용하고, 이를 통해 최종 성능을 확인합니다.

```
...
```

Voting을 통한 앙상블을 만듭니다.

```
from sklearn.ensemble import VotingRegressor
```

```
reg_vt = VotingRegressor([
    #('baseline', reg_lr),
```

```

    #('lr2', reg_lr2),
    ('lr3', reg_lr3),
    ('lr4', reg_lr4),
    ('rf', reg_rf),
    ('xgb', reg_xgb),
])
eval_model('vt', reg_vt, ss)

```

Valid: 0.84565±0.00000, V.Train: 0.84373±0.00000

	result	score
model name		
baseline	Valid: 0.86325±0.00295, V.Train: 0.86300±0.00073	0.863246
lr2	Valid: 0.84697±0.00283, V.Train: 0.84668±0.00071	0.846973
lr3	Valid: 0.84651±0.00256, V.Train: 0.84370±0.00137	0.846513
lr4	Valid: 0.84614±0.00290, V.Train: 0.84329±0.00061	0.846145
rf	Valid: 0.84655±0.00000, V.Train: 0.84634±0.00000	0.846550
vt	Valid: 0.84565±0.00000, V.Train: 0.84373±0.00000	0.845647
xgb	Valid: 0.84615±0.00000, V.Train: 0.84477±0.00000	0.846149

```

model name          vt
result      Valid: 0.84565±0.00000, V.Train: 0.84373±0.00000
score                                0.845647
Name: 6, dtype: object

```

In [24]:

```

# 개선이 있습니다 모델을 선택합니다.
prd = select_model(reg_vt)
# 자가 채점을 해봅니다.
print("Voting 채점 결과:", mean_squared_error(df_ans['target'], prd) ** 0.5)

```

Voting 채점 결과: 0.8475974890236782

3일 간의 실습 강의를 포함하여 총 8일간 강의 들으시느라 고생 많으셨습니다.

궁금하신점 있으시면 연락주세요.

좋은 소식이 들려 오기를 기다리겠습니다.

감사합니다.

멀티캠퍼스 강선구(sunku0316.kang@multicampus.com, sun9sun9@gmail.com) 올림

In []: